

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA TP.HCM
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

PHÁT TRIỂN NỀN TẢNG NHÚNG ĐA CHỨC NĂNG DỰA TRÊN YOCTO PROJECT CHO CÁC ỨNG DỤNG TỰ ĐỘNG HÓA TRONG CÔNG NGHIỆP

HK252

Giảng viên hướng dẫn: TS.Lê Trọng Nhân

STT	Họ & Tên	MSSV	Lớp	Đánh giá
1	Đỗ Tuấn Anh	2210050	MT22KT01	100%
2	Dương Minh Hiếu	2210978	MT22KT01	100%
3	Nguyễn Hạo Duy	2210512	MT22KT01	100%

TP.Hồ Chí Minh, tháng 5 năm 2025



Thông tin hội đồng chấm khóa luận tốt nghiệp

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số 476/QĐĐHCNTT ngày 30 tháng 02 năm 2024 của Hiệu trưởng Trường Đại học Bách Khoa - TP.HCM.

- | | |
|-------------|----------|
| 1. PGS. TS. | Chủ tịch |
| 2. TS. | Ủy viên |
| 3. ThS. | Thư ký |

Lời cảm ơn

Không ai đạt được điều gì đó to lớn mà không nhờ sự giúp đỡ của những người xung quanh, cho dù là trực tiếp hay gián tiếp đi nữa. Để hoàn thành được khóa luận này, tác giả may mắn nhận được nhiều sự giúp đỡ và hỗ trợ từ quý thầy, cô, anh chị, bạn bè và người thân. Tác giả xin dành những trang đầu tiên này để bày tỏ lòng tri ân của mình tới tất cả mọi người, những người đã đồng hành cùng nhóm trong khoảng thời gian vừa qua. Đầu tiên, tác giả xin gửi lời cảm ơn sâu sắc đến toàn thể các thầy cô của Trường Đại học Công nghệ Thông tin nói chung và các thầy cô khoa Mạng máy tính và Truyền thông nói riêng. Nhờ những kiến thức quý giá mà thầy cô đã truyền đạt, cũng như việc hỗ trợ tận tình trong suốt khoảng thời gian thực hiện, nhóm đã hoàn thành khóa luận và đạt được các kết quả đáng ghi nhận. Tác giả xin đặc biệt cảm ơn TS. Google là người đã truyền cảm hứng, tận tình hướng dẫn và hỗ trợ tận tình về kiến thức, tạo môi trường thuận lợi để nhóm có thể học hỏi, trao đổi với các bạn, các em trong nhóm nghiên cứu. Đây là những kiến thức, kinh nghiệm quý giá, không chỉ có tác dụng trong khóa luận tốt nghiệp này mà còn trong khoảng thời gian làm việc trong chặng đường tiếp theo. Cuối cùng, tôi xin bày tỏ lòng tri ân đến gia đình và người thân, những người đã luôn là những hậu phương vững chắc và luôn ủng hộ từng quyết định mà nhóm đưa ra. Mặc dù đã nỗ lực rất nhiều để luận văn được hoàn thiện nhất, song khó có thể tránh khỏi thiếu sót và hạn chế. Kính mong nhận được sự thông cảm và ý kiến đóng góp từ quý thầy cô và các bạn.

Mục Lục

1	Phần mở đầu	7
1.1	Đặt vấn đề	8
1.2	Lý do chọn đề tài	8
1.3	Mục tiêu nghiên cứu	8
1.4	Phạm vi và giới hạn nghiên cứu	8
1.5	Bố cục luận văn	8
2	Tổng quan dự án	9
3	Cơ sở lý thuyết	10
3.1	Tổng quan về Linux nhúng và Yocto Project	11
3.1.1	Giới thiệu về Linux	11
3.1.2	Khái niệm Embedded Linux	12
3.1.3	Giới thiệu Yocto Project và hệ thống Poky	13
3.1.4	Các thành phần cốt lõi: BitBake, Metadata, Layers, Recipes	14
3.2	Kiến trúc Build System trong Yocto Project	15
3.2.1	Kiến trúc Layer trong Yocto	15
3.2.2	Các thành phần chính trong Layer của Yocto Project	16
3.2.3	Recipe trong Yocto	19
3.2.4	BitBake Build Engine	20
3.2.5	Cross-compilation trong Yocto	23
3.3	Linux Device Driver	25
3.3.1	Tổng quan Linux Kernel	25
3.3.2	User Space và Kernel Space	27
3.3.3	Linux Device Driver	29
3.3.4	Device Tree	31
3.3.5	Kernel Module	33
3.3.6	Kernel Logging	35

3.3.7	Kernel Build System	37
3.3.8	Cơ chế tự động nạp module	39
3.3.9	Xây dựng và tích hợp Driver trong Yocto	41
3.4	Bảo mật hệ điều hành Linux nhúng	44
3.5	Edge AI trong hệ thống nhúng	44
3.6	Giao diện người dùng trên thiết bị nhúng	44
3.7	Giao tiếp phần cứng công nghiệp	44
3.7.1	Chuẩn vật lý RS485	44
3.7.2	Giao thức Modbus RTU và cấu trúc khung tin	45
3.7.3	Giao thức truyền tin MQTT/HTTP	47
3.8	Kết nối Cloud và IoT	49
4	Thiết kế hệ thống	50
5	Triển khai và đánh giá	51
6	Sản phẩm demo tích hợp	52
7	Kết luận và hướng phát triển	53

Danh sách bảng

3.1	Các biến cơ bản trong Recipe	19
3.2	Các task chính của BitBake	21
3.3	Mô hình Cross-compilation	24
3.4	So sánh User Space và Kernel Space	28
3.5	Các loại Linux Device Driver	30
3.6	Các lệnh kiểm tra Kernel Module	43

Danh sách hình

3.1	Linux	11
3.2	Yocto Project	13
3.3	Tổng thể hệ sinh thái Yocto Project	14
3.4	RS485 TTL	44
3.5	Modbus RTU	45
3.6	Giao thức MQTT	47
3.7	Giao thức HTTP	48

Chương 1

Phần mở đầu

- 1.1 Đặt vấn đề
- 1.2 Lý do chọn đề tài
- 1.3 Mục tiêu nghiên cứu
- 1.4 Phạm vi và giới hạn nghiên cứu
- 1.5 Bố cục luận văn

1.1 Đặt vấn đề

1.2 Lý do chọn đề tài

1.3 Mục tiêu nghiên cứu

1.4 Phạm vi và giới hạn nghiên cứu

1.5 Bố cục luận văn

Chương 2

Tổng quan dự án

2.1 Tổng quan về tự động hóa công nghiệp

2.2 Tổng quan về hệ điều hành nhúng

2.3 Tổng quan về Yocto Project

2.4 Các công trình liên quan

Chương 3

Cơ sở lý thuyết

- 3.1** Tổng quan về Linux nhúng và Yocto Project
- 3.2** Kiến trúc Build System trong Yocto Project
- 3.3** Linux Device Driver
- 3.4** Bảo mật hệ điều hành Linux nhúng
- 3.5** Edge AI trong hệ thống nhúng
- 3.6** Giao diện người dùng trên thiết bị nhúng
- 3.7** Giao tiếp phần cứng công nghiệp
- 3.8** Kết nối Cloud và IoT

3.1 Tổng quan về Linux nhúng và Yocto Project

3.1.1 Giới thiệu về Linux



Hình 3.1: Linux

Linus Torvalds (sinh ngày 28/12/1969), lớn lên ở Helsinki, Phần Lan. Linus vào Đại học Helsinki năm 1988 và bắt đầu học ngành Khoa học máy tính. Năm 1991, ông bắt đầu khởi xướng dự án LIMS (Linux Is Minix System) - dự án hệ điều hành do ông tự phát triển đánh dấu bước khởi đầu của Linux. Linux được phát triển dựa trên các nguyên lý thiết kế của hệ điều hành Unix, vốn ra đời vào năm 1969. Linux có khả năng chạy trên kiến trúc phần cứng x86 và được phân phối dưới dạng mã nguồn mở. Phiên bản đầu tiên của nhân Linux được công bố công khai vào ngày 17 tháng 9 năm 1991, nhanh chóng thu hút sự quan tâm và đóng góp của cộng đồng lập trình viên trên toàn thế giới.

Mặc dù Linus Torvalds ban đầu phát triển Linux như một dự án cá nhân, trong bối cảnh hệ sinh thái GNU đã cung cấp nhiều công cụ và thư viện hệ thống quan trọng nhưng vẫn thiếu một nhân hệ điều hành hoàn chỉnh, sự ra đời của nhân Linux đã vô tình lấp đầy khoảng trống này, tạo tiền đề cho việc hình thành các hệ thống hoàn chỉnh thường được gọi là GNU/Linux.

Đặc điểm nổi bật của Linux là mô hình phát triển mã nguồn mở phân tán cho phép bất kỳ ai cũng có thể xem, sửa đổi và phân phối. Hiện nay Linux có hàng trăm phiên bản phân phối trên cộng đồng. Một số phiên bản nổi tiếng và phổ biến như: Ubuntu, Red Hat, Debian, Kali, Fedora,... Qua thời gian, Linux đã phát triển từ một dự án cá nhân thành một dự án phần mềm quy mô toàn cầu, với sự tham gia của các cá nhân, cộng đồng và các tập đoàn công nghệ lớn.

Hệ điều hành Linux bao gồm các thành phần chính là Kernel, giao diện dòng lệnh và đồ họa, hệ thống tập tin và quản lý tiến trình.

Linux Kernel là phần lõi của hệ điều hành Linux, chịu trách nhiệm quản lý và điều phối tài nguyên phần cứng của hệ thống, đóng vai trò trung gian giữa phần cứng và các chương trình người dùng. Linux Kernel sử dụng kiến trúc nguyên khối (monolithic). Nghĩa là toàn bộ các chức năng của hệ điều hành hoạt động trong cùng một không gian kernel, tạo nên hiệu suất cao và khả năng kiểm soát chặt chẽ hơn. Một số cơ chế được

nhân Linux cung cấp là bộ lập lịch (scheduler), quản lý bộ nhớ, device drivers, hỗ trợ giao tiếp liên tiến trình (IPC) và mạng.

Giao diện dòng lệnh (CLI) và đồ hoạ cho phép người dùng tương tác trực tiếp với hệ điều hành thông qua các lệnh văn bản. Giao diện dòng lệnh trong Linux thường được cung cấp bởi các shell như Bash, Zsh hoặc Fish. CLI cho phép người dùng thực thi lệnh hệ thống, viết scripts và quản trị hệ thống một cách có hiệu quả. Bên cạnh đó, giao diện đồ hoạ cung cấp môi trường desktop, giúp người dùng dễ dàng thao tác và sử dụng hệ thống trong các ứng dụng thông thường.

Hệ thống tập tin trong Linux được tổ chức theo một hệ thống phân bậc như cấu trúc của một cây phân cấp với bậc cao nhất được kí hiệu là "/". Linux hỗ trợ nhiều loại hệ thống tập tin khác nhau như ext4, XFS, Btrfs,... cho phép lưu trữ và truy xuất dữ liệu một cách hiệu quả. Mọi tài nguyên trong hệ thống đều được biểu diễn thống nhất dưới dạng tập tin, góp phần đơn giản hoá việc quản lý.

Quản lý tiến trình nhằm điều phối việc thực thi các chương trình trong hệ thống. Mỗi chương trình đang chạy được coi là một tiến trình với không gian địa chỉ và tài nguyên riêng biệt. Linux Kernel lập lịch để phân chia thời gian xử lý của CPU cho các tiến trình, đảm bảo khả năng đa nhiệm và hiệu suất cho hệ thống.

Ngày nay, Linux được sử dụng phổ biến và rộng rãi không chỉ trên các máy chủ, máy tính cá nhân mà còn góp mặt trong các hệ thống nhúng, thiết bị di động, các hạ tầng điện toán đám mây,... Sự phát triển liên tục của Linux khẳng định vai trò như một trong những nền tảng hệ điều hành quan trọng nhất trong lịch sử công nghệ thông tin hiện đại.

3.1.2 Khái niệm Embedded Linux

Embedded Linux hay còn gọi là Linux Nhúng là một thuật ngữ dùng để chỉ các hệ thống nhúng, trong đó nhân Linux (Linux kernel) được sử dụng làm lõi hệ điều hành cho các thiết bị có chức năng chuyên biệt. Khác với các hệ điều hành Linux trên máy tính đa năng, Embedded Linux chỉ giữ lại những thành phần thật sự cần dùng, giúp hệ thống hoạt động hiệu quả, tiết kiệm tài nguyên và phù hợp với các yêu cầu đặc thù của thiết bị nhúng.

Đặc trưng của Embedded Linux là khả năng tùy biến cao cho phép nhà phát triển có thể điều chỉnh cấu hình kernel hay có thể loại bỏ các dịch vụ không cần thiết và chỉ giữ lại các mô-đun phù hợp với chức năng của hệ thống. Cách tiếp cận này giúp giảm kích thước hệ thống, rút ngắn thời gian khởi động và nâng cao độ ổn định trong quá trình vận hành. Đồng thời, Embedded Linux vẫn kế thừa các ưu điểm cốt lõi của Linux như khả năng quản lý tiến trình, hỗ trợ đa nhiệm, đa luồng và khả năng mở rộng trên nhiều kiến trúc phần cứng khác nhau.

Ngày nay, Embedded Linux thường được triển khai trên các thiết bị nhúng hiện đại sử dụng kiến trúc vi xử lý tiết kiệm năng lượng, nơi hệ thống phải đáp ứng các yêu cầu vận hành liên tục, độ tin cậy cao và trong nhiều trường hợp là các ràng buộc về thời gian thực. Nhờ khả năng cấu hình linh hoạt và hệ sinh thái phần mềm phong phú, Embedded Linux trở thành một nền tảng phổ biến trong việc phát triển các thiết bị thông minh, thiết bị mạng, hệ thống điều khiển công nghiệp và các ứng dụng nhúng phức tạp.

3.1.3 Giới thiệu Yocto Project và hệ thống Poky



Hình 3.2: Yocto Project

Yocto Project là một dự án mã nguồn mở mang tính cộng tác toàn cầu, được thiết kế để hỗ trợ các nhà phát triển xây dựng hệ thống Linux tùy biến cho các thiết bị nhúng, không phụ thuộc vào kiến trúc phần cứng. Thay vì cung cấp một bản phân phối Linux cố định, Yocto Project cung cấp bộ công cụ, môi trường phát triển và các quy tắc xây dựng, cho phép tạo ra các image Linux được tối ưu riêng cho từng sản phẩm nhúng.

Yocto Project đã trở thành một chuẩn công nghiệp trong việc xây dựng các bộ phần mềm nhúng nhờ khả năng tái sử dụng, mở rộng và bảo trì lâu dài trên nhiều nền tảng phần cứng. Dự án không phụ thuộc vào kiến trúc phần cứng, hỗ trợ đa dạng nền tảng thông qua các gói BSP, đồng thời cho phép xây dựng các bản phân phối Linux tùy biến phù hợp với nhiều dòng sản phẩm. Nhờ cơ chế chỉ tích hợp những thành phần cần thiết, Yocto Project đặc biệt phù hợp với các thiết bị nhúng có tài nguyên hạn chế, giúp tối ưu dung lượng và hiệu năng. Ngoài ra, dự án còn cung cấp hệ thống công cụ phát triển đầy đủ, mô hình Layer rõ ràng, khả năng tái tạo nhị phân cao và hỗ trợ quản lý giấy phép phần mềm, đáp ứng tốt các yêu cầu phát triển và thương mại hóa sản phẩm.

Poky là bản phân phối tham chiếu của Yocto Project, cung cấp một bộ khung chuẩn để xây dựng các hệ thống Linux nhúng tùy biến. Poky tích hợp OpenEmbedded Build System, bao gồm BitBake và OpenEmbedded-Core, cùng với các lớp metadata cần thiết để cấu hình, biên dịch và tạo image Linux từ mã nguồn. Đây không phải là một hệ điều hành hoàn chỉnh dùng trực tiếp cho sản phẩm, mà là điểm khởi đầu đã được kiểm thử và đảm bảo tính tương thích, cho phép nhà phát triển mở rộng và tùy biến hệ thống

thông qua cơ chế layer để phù hợp với các yêu cầu phần cứng và chức năng cụ thể của thiết bị nhúng.

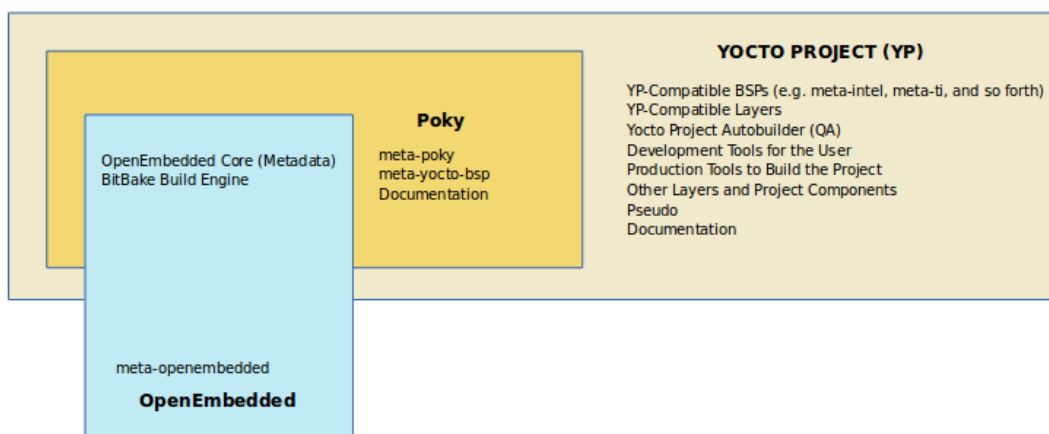
3.1.4 Các thành phần cốt lõi: BitBake, Metadata, Layers, Recipes

BitBake là thành phần trung tâm của hệ thống xây dựng trong Yocto Project, đóng vai trò như một bộ lập lịch và thực thi tác vụ. BitBake phân tích các chỉ dẫn được mô tả trong metadata, xây dựng cây phụ thuộc giữa các thành phần và điều phối quá trình biên dịch, đóng gói để tạo ra image Linux nhúng theo cấu hình đã định.

Metadata là tập hợp các thông tin điều khiển toàn bộ quá trình xây dựng hệ thống, bao gồm các tệp cấu hình, lớp và công thức xây dựng. Metadata quyết định những thành phần phần mềm nào được sử dụng, cách thức chúng được cấu hình, cũng như các chính sách ảnh hưởng đến quá trình build, từ đó cho phép kiểm soát chặt chẽ và nhất quán hệ thống tạo ra.

Layers là các lớp metadata được tổ chức theo mô-đun, mỗi layer đại diện cho một nhóm chức năng hoặc phạm vi cụ thể như phần cứng, middleware hay ứng dụng. Cơ chế layer cho phép mở rộng, ghi đè và tùy biến hệ thống một cách linh hoạt, đồng thời giúp giảm độ phức tạp và tăng khả năng tái sử dụng trong quá trình phát triển.

Recipes là đơn vị cơ bản của metadata, mô tả cách thức tải mã nguồn, áp dụng bản vá, cấu hình, biên dịch và đóng gói từng gói phần mềm. Thông qua các recipe, BitBake có thể tự động xây dựng các thành phần riêng lẻ và tích hợp chúng thành một hệ thống Linux nhúng hoàn chỉnh.



Hình 3.3: Tổng thể hệ sinh thái Yocto Project

3.2 Kiến trúc Build System trong Yocto Project

3.2.1 Kiến trúc Layer trong Yocto

Yocto Project sử dụng kiến trúc phân lớp (Layer Architecture) nhằm tổ chức metadata và mã nguồn theo hướng module hóa, linh hoạt và dễ mở rộng. Đây là một trong những đặc điểm quan trọng giúp Yocto trở thành nền tảng xây dựng Linux Embedded có khả năng tùy biến cao và phù hợp với nhiều kiến trúc phần cứng khác nhau.

Trong các hệ thống Linux Embedded hiện đại, số lượng package, driver, thư viện và thành phần hệ thống thường rất lớn. Nếu toàn bộ metadata và source code được lưu tập trung trong một cấu trúc duy nhất, việc quản lý dependency, cập nhật phiên bản và bảo trì hệ thống sẽ trở nên phức tạp. Do đó, Yocto áp dụng mô hình phân lớp nhằm chia nhỏ hệ thống thành các thành phần độc lập, mỗi layer đảm nhiệm một nhóm chức năng riêng biệt.

Kiến trúc phân lớp cho phép tách biệt rõ ràng giữa các thành phần hệ thống như BSP (Board Support Package), middleware, application và các package tùy chỉnh. Nhờ đó, người phát triển có thể dễ dàng bổ sung hoặc loại bỏ một chức năng mà không ảnh hưởng đến các thành phần còn lại của hệ thống. Điều này đặc biệt quan trọng trong phát triển hệ thống nhúng, nơi mỗi nền tảng phần cứng có thể yêu cầu các driver và cấu hình riêng biệt.

Ngoài khả năng module hóa, Layer Architecture còn giúp tăng tính tái sử dụng của metadata và source code. Một layer có thể được sử dụng lại trong nhiều dự án khác nhau mà không cần chỉnh sửa đáng kể. Ví dụ, các layer hỗ trợ kiến trúc phần cứng, networking hoặc multimedia có thể được tích hợp vào nhiều hệ thống Linux Embedded khác nhau thông qua cơ chế add-layer của Yocto.

Bên cạnh đó, việc phân lớp còn hỗ trợ hiệu quả cho quá trình cộng tác và phát triển nhóm. Mỗi nhóm phát triển có thể quản lý một layer riêng biệt tương ứng với chức năng của mình mà không làm ảnh hưởng trực tiếp đến hệ thống tổng thể. Điều này giúp giảm xung đột mã nguồn và nâng cao khả năng bảo trì dự án trong các hệ thống quy mô lớn.

Một ưu điểm quan trọng khác của kiến trúc phân lớp là khả năng quản lý ưu tiên metadata thông qua cơ chế layer priority. Khi nhiều layer cùng chứa recipe hoặc cấu hình cho một package, BitBake sẽ xác định layer có mức ưu tiên cao hơn để sử dụng. Cơ chế này cho phép người phát triển tùy chỉnh hoặc ghi đè hành vi của package mà không cần chỉnh sửa trực tiếp source code gốc.

Ngoài ra, Layer Architecture còn giúp Yocto Project duy trì tính mở rộng và khả năng nâng cấp hệ thống trong thời gian dài. Khi cần cập nhật phiên bản Linux Kernel, thêm driver mới hoặc tích hợp middleware khác, người phát triển chỉ cần bổ sung hoặc

cập nhật layer tương ứng thay vì chỉnh sửa toàn bộ hệ thống build.

Nhờ các ưu điểm về tính module hóa, khả năng tái sử dụng, dễ bảo trì và hỗ trợ mở rộng hệ thống, kiến trúc phân lớp đã trở thành nền tảng cốt lõi trong thiết kế của Yocto Project và đóng vai trò quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại. Mỗi layer có thể chứa:

- Recipe (.bb)
- Configuration
- Patch
- Source code

3.2.2 Các thành phần chính trong Layer của Yocto Project

Trong Yocto Project, kiến trúc layer được xây dựng dựa trên cơ chế metadata nhằm tổ chức hệ thống build theo hướng module hóa và có khả năng mở rộng cao. Mỗi layer đóng vai trò như một đơn vị chức năng độc lập, có thể chứa đầy đủ các thành phần cần thiết để xây dựng phần mềm hoặc hệ thống Linux nhúng. Việc phân chia các thành phần trong layer giúp quá trình phát triển, bảo trì và tái sử dụng mã nguồn trở nên thuận tiện hơn. Các thành phần chính thường xuất hiện trong một layer bao gồm Recipe, Configuration, Patch và Source Code.

Recipe (.bb)

Recipe là thành phần cốt lõi trong Yocto Project, được sử dụng để mô tả toàn bộ quy trình xây dựng một package hoặc một thành phần phần mềm trong hệ thống nhúng. Các recipe thường có phần mở rộng `.bb` và được xử lý bởi BitBake Build Engine. Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn hệ thống build thực hiện các công việc như tải mã nguồn, giải nén, biên dịch, cài đặt và đóng gói phần mềm.

Trong mỗi recipe, nhiều biến hệ thống được sử dụng để mô tả thông tin liên quan đến package. Các biến này bao gồm tên package, phiên bản, giấy phép sử dụng, dependency, vị trí source code và các task cần thực hiện trong quá trình build. Thông qua recipe, BitBake có thể tự động phân tích dependency giữa các package và xây dựng quy trình biên dịch phù hợp.

Ngoài vai trò quản lý build process, recipe còn hỗ trợ tính module hóa và tái sử dụng trong Yocto Project. Thay vì cấu hình thủ công từng bước biên dịch, người phát triển chỉ cần mô tả metadata trong recipe để hệ thống tự động thực hiện toàn bộ pipeline build. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển hệ thống Linux nhúng quy mô lớn.

Recipe cũng hỗ trợ kế thừa thông qua cơ chế class inheritance. Nhờ đó, nhiều package có thể sử dụng chung một tập chức năng build mà không cần lặp lại mã cấu hình. Đây là một trong những đặc điểm quan trọng giúp Yocto Project trở thành nền tảng build có tính mở rộng và bảo trì cao.

Configuration

Configuration là thành phần chịu trách nhiệm quản lý và thiết lập môi trường build trong Yocto Project. Các file cấu hình thường được lưu trong thư mục `conf/` và chứa các biến hệ thống phục vụ quá trình xây dựng Linux Embedded. Đây là thành phần quan trọng giúp xác định cách thức hoạt động của toàn bộ hệ thống build.

Thông qua configuration, người phát triển có thể thiết lập kiến trúc phần cứng mục tiêu, compiler, toolchain, package manager, loại filesystem và nhiều tham số hệ thống khác. Ngoài ra, configuration còn cho phép quản lý danh sách layer được sử dụng trong môi trường build cũng như thứ tự ưu tiên giữa các layer.

Một trong những vai trò quan trọng của configuration là hỗ trợ khả năng tùy biến hệ thống Linux nhúng. Tùy thuộc vào yêu cầu phần cứng và ứng dụng, người phát triển có thể thay đổi các thông số cấu hình để tạo ra các image Linux tối ưu về kích thước, hiệu năng hoặc mức tiêu thụ tài nguyên.

Các file configuration trong Yocto thường được tổ chức theo nhiều cấp độ khác nhau như cấu hình toàn cục, cấu hình machine, cấu hình distro hoặc cấu hình riêng cho từng layer. Nhờ kiến trúc này, Yocto Project có khả năng quản lý các hệ thống build phức tạp với số lượng package và dependency rất lớn.

Ngoài chức năng quản lý môi trường build, configuration còn đóng vai trò quan trọng trong việc đảm bảo tính tái lập của hệ thống. Khi các thông số build được định nghĩa rõ ràng trong configuration, hệ thống có thể được build lại với kết quả tương đồng trên nhiều môi trường phát triển khác nhau.

Patch

Patch là thành phần được sử dụng để sửa đổi hoặc cập nhật mã nguồn trong quá trình build mà không cần thay đổi trực tiếp source code gốc. Trong Yocto Project, patch đóng vai trò quan trọng trong việc tùy biến phần mềm, sửa lỗi hệ thống và bổ sung chức năng mới cho các package.

Patch thường được lưu dưới dạng file `.patch` hoặc `.diff`. Các file này chứa thông tin thay đổi giữa phiên bản mã nguồn gốc và phiên bản đã được chỉnh sửa. Trong quá trình build, BitBake sẽ tự động áp dụng patch trước khi thực hiện biên dịch mã nguồn.

Việc sử dụng patch mang lại nhiều lợi ích trong phát triển hệ thống nhúng. Trước hết, patch giúp duy trì tính nguyên vẹn của source code gốc, từ đó tạo thuận lợi cho việc

cập nhật phiên bản mới hoặc đồng bộ với repository chính thức. Bên cạnh đó, patch còn hỗ trợ quản lý thay đổi một cách rõ ràng và có hệ thống, giúp quá trình bảo trì phần mềm trở nên dễ dàng hơn.

Trong các hệ thống Linux Embedded, patch thường được sử dụng để tối ưu kernel, sửa lỗi driver, tùy biến bootloader hoặc bổ sung hỗ trợ cho phần cứng mới. Đây là kỹ thuật phổ biến trong phát triển phần mềm nhúng vì nhiều thành phần hệ thống thường cần được tùy chỉnh để phù hợp với nền tảng phần cứng cụ thể.

Ngoài ra, patch còn đóng vai trò quan trọng trong việc quản lý vòng đời phần mềm. Thay vì chỉnh sửa trực tiếp source code, toàn bộ thay đổi được lưu trữ dưới dạng patch giúp dễ dàng theo dõi lịch sử chỉnh sửa, rollback hoặc áp dụng lại trên các phiên bản hệ thống khác nhau.

Source Code

Source Code là thành phần chứa mã nguồn của chương trình hoặc hệ thống phần mềm được sử dụng trong quá trình build. Đây là thành phần cốt lõi quyết định chức năng và hành vi của package trong hệ thống Linux Embedded.

Trong Yocto Project, source code có thể được lấy từ nhiều nguồn khác nhau như local file, Git repository, tarball package hoặc remote server. Hệ thống BitBake sẽ tự động tải và quản lý source code dựa trên thông tin được khai báo trong recipe.

Source code trong hệ thống nhúng thường bao gồm nhiều loại thành phần như application, library, kernel module, bootloader hoặc device driver. Tùy thuộc vào chức năng của package, source code có thể được viết bằng các ngôn ngữ như C, C++, Python hoặc Shell Script.

Một trong những đặc điểm quan trọng của source code trong Linux Embedded là khả năng tương tác trực tiếp với phần cứng hệ thống. Điều này đòi hỏi mã nguồn phải được tối ưu về hiệu năng, khả năng quản lý tài nguyên và độ ổn định trong quá trình vận hành.

Ngoài chức năng thực thi logic hệ thống, source code còn đóng vai trò quan trọng trong việc đảm bảo tính mở rộng và khả năng bảo trì phần mềm. Việc tổ chức mã nguồn theo kiến trúc module giúp hệ thống dễ dàng nâng cấp, tái sử dụng và tích hợp thêm các chức năng mới trong tương lai.

Trong các dự án Linux Embedded quy mô lớn, source code thường được quản lý thông qua hệ thống version control như Git nhằm hỗ trợ cộng tác nhóm, theo dõi lịch sử thay đổi và quản lý phiên bản phần mềm một cách hiệu quả.

3.2.3 Recipe trong Yocto

Trong Yocto Project, Recipe là thành phần trung tâm của hệ thống build và đóng vai trò mô tả toàn bộ quy trình xây dựng một package phần mềm. Recipe thường có phần mở rộng `.bb` (BitBake Recipe) và được BitBake sử dụng để thực hiện các tác vụ như tải mã nguồn, cấu hình môi trường build, biên dịch, cài đặt và đóng gói package.

Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn BitBake cách xây dựng một thành phần phần mềm cụ thể trong hệ thống Linux Embedded. Thông qua recipe, Yocto Project có khả năng tự động hóa toàn bộ quy trình build và quản lý dependency giữa các package trong hệ thống.

Một recipe thường chứa nhiều biến và task nhằm mô tả thông tin liên quan đến package. Các biến này được sử dụng để khai báo metadata như tên package, phiên bản, license, vị trí source code và phương thức build. Trong khi đó, các task chịu trách nhiệm thực thi từng giai đoạn trong pipeline build của BitBake.

Một số biến quan trọng trong recipe được trình bày trong Bảng 3.1.

Biến	Ý nghĩa
SUMMARY	Mô tả package
LICENSE	Khai báo license
SRC_URI	Đường dẫn source code
S	Thư mục source
inherit	Kế thừa class

Bảng 3.1: Các biến cơ bản trong Recipe

Biến `SUMMARY` được sử dụng để mô tả ngắn gọn chức năng của package. Thông tin này giúp người phát triển dễ dàng nhận biết vai trò của package trong hệ thống build.

Biến `LICENSE` dùng để khai báo giấy phép sử dụng phần mềm. Việc quản lý license là yêu cầu quan trọng trong các hệ thống Linux Embedded nhằm đảm bảo tuân thủ các điều kiện sử dụng phần mềm mã nguồn mở.

Biến `SRC_URI` được sử dụng để xác định vị trí source code hoặc các file cần thiết cho quá trình build. Source code có thể được lấy từ nhiều nguồn khác nhau như local file, Git repository, tarball package hoặc remote server.

Biến `S` xác định thư mục chứa source code sau khi quá trình giải nén hoàn tất. Biến này giúp BitBake xác định chính xác vị trí thực hiện các task biên dịch và cài đặt.

Biến `inherit` cho phép recipe kế thừa các class build có sẵn trong Yocto Project. Cơ chế kế thừa giúp tái sử dụng các chức năng build phổ biến và giảm sự trùng lặp trong metadata.

Trong quá trình build, BitBake sẽ phân tích dependency giữa các recipe để xác định thứ tự thực thi phù hợp. Nhờ đó, hệ thống có thể tự động build toàn bộ Linux Embedded

Image với số lượng package lớn mà vẫn đảm bảo tính nhất quán.

Một đặc điểm quan trọng của recipe là khả năng tái sử dụng và mở rộng thông qua cơ chế append và class inheritance. Người phát triển có thể bổ sung hoặc ghi đè metadata của recipe mà không cần chỉnh sửa trực tiếp file gốc. Điều này giúp quá trình bảo trì hệ thống trở nên dễ dàng hơn, đặc biệt trong các dự án Linux Embedded quy mô lớn.

Ngoài ra, recipe còn đóng vai trò quan trọng trong việc đảm bảo tính tái lập của hệ thống build. Khi toàn bộ metadata và quy trình build được định nghĩa rõ ràng trong recipe, hệ thống có thể được build lại trên nhiều môi trường phát triển khác nhau với kết quả tương đồng.

Nhờ khả năng tự động hóa, quản lý dependency và hỗ trợ tùy biến mạnh mẽ, recipe đã trở thành thành phần cốt lõi trong kiến trúc của Yocto Project và là nền tảng quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại.

3.2.4 BitBake Build Engine

BitBake là build engine trung tâm của Yocto Project, chịu trách nhiệm phân tích metadata và thực thi các task được định nghĩa trong recipe nhằm xây dựng package hoặc toàn bộ hệ thống Linux Embedded. Có thể xem BitBake như một hệ thống tự động hóa build hoạt động dựa trên metadata, cho phép quản lý dependency, thực thi pipeline build và tạo ra image Linux hoàn chỉnh cho nền tảng nhúng.

Trong Yocto Project, mỗi recipe sẽ được BitBake phân tích để xác định các task cần thực hiện cũng như mối quan hệ phụ thuộc giữa các package. Dựa trên dependency graph, BitBake sẽ tự động xác định thứ tự thực thi phù hợp nhằm đảm bảo quá trình build diễn ra chính xác và nhất quán.

Một trong những ưu điểm quan trọng của BitBake là khả năng tự động hóa toàn bộ quá trình build. Thay vì thực hiện thủ công từng bước như tải source code, cấu hình compiler hoặc cài đặt package, BitBake cho phép toàn bộ pipeline build được mô tả thông qua metadata trong recipe. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển hệ thống Linux Embedded.

Trong quá trình build, BitBake sử dụng tập hợp các task để xử lý từng giai đoạn khác nhau. Mỗi task đại diện cho một bước cụ thể trong pipeline build và được thực thi theo trình tự xác định.

Các task chính của BitBake được trình bày trong Bảng 3.2.

Task	Chức năng
do_fetch	Tải source code
do_unpack	Giải nén source
do_patch	Áp dụng patch
do_compile	Biên dịch mã nguồn
do_install	Cài đặt package
do_package	Đóng gói package

Bảng 3.2: Các task chính của BitBake

Task do_fetch

Task `do_fetch` là giai đoạn đầu tiên trong pipeline build của BitBake. Nhiệm vụ chính của task này là tải source code hoặc các tài nguyên cần thiết phục vụ quá trình build package.

Source code có thể được lấy từ nhiều nguồn khác nhau như Git repository, HTTP server, FTP server, local file hoặc tarball package. BitBake sử dụng thông tin trong biến `SRC_URI` để xác định vị trí dữ liệu cần tải.

Trong quá trình fetch, BitBake cũng thực hiện kiểm tra checksum nhằm đảm bảo tính toàn vẹn của source code. Điều này giúp hạn chế lỗi build do source code bị thay đổi hoặc hư hỏng trong quá trình tải xuống.

Task `do_fetch` đóng vai trò quan trọng trong việc đảm bảo khả năng tái lập của hệ thống build. Khi source code được quản lý thông qua metadata, cùng một recipe có thể được sử dụng để build lại package trên nhiều môi trường khác nhau với kết quả tương đồng.

Task do_unpack

Sau khi source code được tải về, BitBake thực hiện task `do_unpack` nhằm giải nén và chuẩn bị mã nguồn cho các bước xử lý tiếp theo.

Trong giai đoạn này, các file nén như `.tar.gz`, `.zip` hoặc package source sẽ được giải nén vào thư mục làm việc của BitBake. Đối với source code được tải từ Git repository, task này sẽ thực hiện checkout source code vào workspace tương ứng.

Task `do_unpack` giúp chuẩn hóa cấu trúc source code trước khi chuyển sang các bước patch hoặc biên dịch. Điều này đảm bảo các task phía sau luôn làm việc trên một môi trường source code thống nhất.

Ngoài chức năng giải nén, task này còn hỗ trợ quản lý workspace build một cách tự động, giúp người phát triển không cần thao tác thủ công với source code trong quá trình build hệ thống.

Task do_patch

Task `do_patch` được sử dụng để áp dụng các patch lên source code sau khi quá trình giải nén hoàn tất. Đây là bước quan trọng trong việc tùy biến package hoặc sửa lỗi hệ thống mà không cần thay đổi trực tiếp mã nguồn gốc.

Patch thường được sử dụng trong các trường hợp:

- Sửa lỗi phần mềm.
- Tùy biến kernel hoặc driver.
- Bổ sung tính năng mới.
- Tối ưu hệ thống cho phần cứng cụ thể.

Trong quá trình thực thi task `do_patch`, BitBake sẽ tự động tìm và áp dụng các file patch được khai báo trong recipe. Nếu patch không thể áp dụng thành công, quá trình build sẽ dừng lại để tránh tạo ra package không nhất quán.

Việc sử dụng task patch giúp duy trì tính nguyên vẹn của source code gốc, đồng thời hỗ trợ quản lý thay đổi phần mềm một cách có hệ thống và dễ bảo trì hơn.

Task do_compile

Task `do_compile` chịu trách nhiệm biên dịch source code thành mã máy hoặc binary executable có thể sử dụng trên hệ thống đích.

Trong hệ thống Linux Embedded, quá trình compile thường sử dụng cross-compiler nhằm tạo binary phù hợp với kiến trúc phần cứng mục tiêu như ARM, x86 hoặc RISC-V. BitBake sẽ tự động thiết lập compiler, sysroot và môi trường build dựa trên metadata của hệ thống.

Task `do_compile` có thể thực hiện nhiều loại build khác nhau như:

- Build application.
- Build library.
- Build kernel module.
- Build Linux Kernel.

Trong quá trình compile, BitBake cũng quản lý dependency giữa các package nhằm đảm bảo các thư viện hoặc thành phần cần thiết đã được build trước đó.

Đây là một trong những task quan trọng nhất trong pipeline build vì nó quyết định trực tiếp đến khả năng hoạt động của package trên hệ thống đích.

Task do_install

Sau khi quá trình biên dịch hoàn tất, BitBake thực hiện task `do_install` để cài đặt các file output vào thư mục staging hoặc root filesystem tạm thời.

Trong giai đoạn này, các binary executable, library, configuration file và resource file sẽ được sắp xếp theo đúng cấu trúc filesystem chuẩn của Linux. Điều này giúp package có thể được tích hợp vào root filesystem của Embedded Linux Image.

Task `do_install` đóng vai trò quan trọng trong việc chuẩn hóa cấu trúc package trước khi đóng gói. Nếu quá trình install không đúng cấu trúc, package có thể không hoạt động chính xác trên hệ thống runtime.

Ngoài ra, task này còn hỗ trợ việc tách package thành nhiều thành phần khác nhau như runtime package, development package hoặc debug package nhằm tối ưu quá trình quản lý phần mềm trong Linux Embedded.

Task do_package

Task `do_package` là giai đoạn cuối trong pipeline build của BitBake, chịu trách nhiệm đóng gói các file đã được cài đặt thành package có thể triển khai trên hệ thống đích.

BitBake hỗ trợ nhiều định dạng package khác nhau như:

- RPM
- DEB
- IPK

Trong quá trình đóng gói, BitBake sẽ tự động phân tích dependency runtime giữa các package nhằm đảm bảo hệ thống có thể cài đặt và vận hành chính xác.

Task `do_package` giúp chuẩn hóa quá trình quản lý phần mềm trong Linux Embedded, đồng thời hỗ trợ cập nhật và bảo trì hệ thống một cách linh hoạt hơn.

Ngoài ra, các package được tạo ra từ task này còn có thể được tái sử dụng trong nhiều image khác nhau mà không cần build lại source code, góp phần tối ưu thời gian build và nâng cao hiệu quả phát triển hệ thống nhúng.

3.2.5 Cross-compilation trong Yocto

Trong các hệ thống Linux Embedded, kiến trúc phần cứng của thiết bị đích thường khác với kiến trúc của máy tính dùng để phát triển phần mềm. Do đó, quá trình biên dịch mã nguồn không thể thực hiện trực tiếp bằng compiler thông thường trên hệ thống

host. Để giải quyết vấn đề này, Yocto Project sử dụng cơ chế Cross-compilation nhằm hỗ trợ biên dịch phần mềm cho nhiều nền tảng phần cứng khác nhau.

Cross-compilation là quá trình biên dịch mã nguồn trên một hệ thống máy tính nhưng chương trình được tạo ra sẽ chạy trên một hệ thống khác có kiến trúc phần cứng khác biệt. Trong Linux Embedded, quá trình này thường diễn ra khi source code được build trên máy tính x86 nhưng binary output được thiết kế để chạy trên thiết bị ARM, RISC-V hoặc các nền tảng nhúng chuyên dụng khác.

Cơ chế cross-compilation đóng vai trò rất quan trọng trong phát triển hệ thống nhúng vì phần lớn thiết bị embedded có tài nguyên hạn chế, không đủ khả năng thực hiện quá trình biên dịch trực tiếp trên thiết bị. Việc build trên máy host giúp tận dụng hiệu năng xử lý cao của máy tính phát triển, từ đó giảm đáng kể thời gian build và tối ưu quy trình phát triển phần mềm.

Trong mô hình cross-compilation, hệ thống thường bao gồm ba thành phần chính: Build Machine, Host Machine và Target Machine. Mỗi thành phần đảm nhiệm một vai trò khác nhau trong quá trình xây dựng phần mềm.

Các thành phần trong mô hình cross-compilation được trình bày trong Bảng 3.3.

Thành phần	Vai trò
Build Machine	Máy thực hiện build
Host Machine	Máy chạy toolchain
Target Machine	Thiết bị nhúng đích

Bảng 3.3: Mô hình Cross-compilation

Build Machine là hệ thống thực hiện toàn bộ quá trình build package hoặc image Linux Embedded. Đây thường là máy tính cá nhân hoặc server phát triển có hiệu năng xử lý cao và chạy hệ điều hành Linux. Build Machine chịu trách nhiệm thực thi BitBake, quản lý metadata và điều phối toàn bộ pipeline build trong Yocto Project.

Host Machine là hệ thống chứa và thực thi toolchain dùng cho quá trình cross-compilation. Trong nhiều trường hợp, Build Machine và Host Machine có thể là cùng một thiết bị vật lý. Tuy nhiên, về mặt kiến trúc logic, Host Machine đại diện cho môi trường chạy compiler và các công cụ build cần thiết để tạo binary cho hệ thống đích.

Target Machine là thiết bị nhúng đích nơi chương trình sau khi biên dịch sẽ được triển khai và thực thi. Đây có thể là board ARM, gateway công nghiệp, hệ thống IoT hoặc các nền tảng embedded chuyên dụng khác. Kiến trúc phần cứng của Target Machine thường khác với kiến trúc của Build Machine, do đó cần sử dụng cross-compiler phù hợp để tạo binary tương thích.

Trong Yocto Project, cross-compilation được quản lý tự động thông qua hệ thống metadata và toolchain. BitBake sẽ tự động lựa chọn compiler, linker và thư viện phù

hợp với kiến trúc của target device dựa trên cấu hình machine và distro. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển Linux Embedded.

Một thành phần quan trọng trong cross-compilation là cross-toolchain. Toolchain là tập hợp các công cụ phục vụ quá trình build như compiler, assembler, linker và debugger. Trong hệ thống embedded, toolchain thường được thiết kế riêng cho từng kiến trúc phần cứng nhằm đảm bảo binary output có thể hoạt động chính xác trên thiết bị đích.

Ngoài compiler, Yocto còn sử dụng sysroot để hỗ trợ cross-compilation. Sysroot là môi trường filesystem chứa các header file, library và dependency cần thiết cho quá trình build. Việc sử dụng sysroot giúp compiler truy cập đúng thư viện và tránh xung đột giữa môi trường host và target.

Một ưu điểm quan trọng của cross-compilation trong Yocto là khả năng tự động hóa và tái lập hệ thống build. Khi toàn bộ toolchain và dependency được quản lý thông qua metadata, hệ thống có thể được build lại trên nhiều môi trường khác nhau mà vẫn đảm bảo tính nhất quán của binary output.

Ngoài ra, cơ chế cross-compilation còn giúp tối ưu hiệu năng phát triển phần mềm nhúng. Thay vì biên dịch trực tiếp trên thiết bị embedded có tài nguyên hạn chế, quá trình build được thực hiện trên máy tính hiệu năng cao, từ đó rút ngắn thời gian build và tăng hiệu quả phát triển hệ thống.

Nhờ khả năng hỗ trợ đa kiến trúc phần cứng, tự động quản lý toolchain và tối ưu quy trình build, cross-compilation đã trở thành thành phần cốt lõi trong Yocto Project và đóng vai trò quan trọng trong quá trình phát triển các hệ thống Linux Embedded hiện đại.

3.3 Linux Device Driver

3.3.1 Tổng quan Linux Kernel

Linux Kernel là thành phần lõi của hệ điều hành Linux, đóng vai trò trung gian giữa phần cứng và các ứng dụng người dùng. Kernel chịu trách nhiệm quản lý tài nguyên hệ thống, điều phối hoạt động của phần cứng và cung cấp các dịch vụ cơ bản cho chương trình ứng dụng thông qua cơ chế system call.

Trong kiến trúc hệ điều hành Linux, kernel hoạt động ở mức đặc quyền cao nhất của hệ thống và có quyền truy cập trực tiếp đến tài nguyên phần cứng như CPU, bộ nhớ, thiết bị ngoại vi và hệ thống lưu trữ. Nhờ đó, kernel có thể kiểm soát toàn bộ hoạt động của hệ thống và đảm bảo các tiến trình hoạt động ổn định, an toàn và hiệu quả.

Linux sử dụng kiến trúc Monolithic Kernel, trong đó nhiều thành phần như driver, memory manager, scheduler và file system được tích hợp trực tiếp trong không gian kernel space. Kiến trúc này giúp Linux đạt hiệu năng cao do các thành phần hệ thống

có thể giao tiếp trực tiếp với nhau mà không cần thông qua nhiều lớp trung gian.

Một trong những vai trò quan trọng nhất của Linux Kernel là quản lý tiến trình (Process Management). Kernel chịu trách nhiệm tạo, lập lịch và hủy tiến trình trong hệ thống. Thông qua bộ lập lịch (Scheduler), kernel quyết định tiến trình nào sẽ được cấp phát CPU tại từng thời điểm nhằm tối ưu hiệu năng xử lý và đảm bảo tính công bằng giữa các tiến trình.

Trong môi trường đa nhiệm, nhiều tiến trình có thể hoạt động đồng thời trên hệ thống. Kernel sử dụng cơ chế context switching để chuyển đổi CPU giữa các tiến trình khác nhau một cách nhanh chóng. Điều này giúp hệ điều hành có khả năng thực thi nhiều tác vụ đồng thời mà vẫn đảm bảo tính ổn định và hiệu quả sử dụng tài nguyên.

Ngoài quản lý tiến trình, Linux Kernel còn chịu trách nhiệm quản lý bộ nhớ (Memory Management). Chức năng này bao gồm cấp phát, thu hồi và bảo vệ vùng nhớ cho các tiến trình trong hệ thống. Kernel sử dụng cơ chế Virtual Memory nhằm tạo không gian địa chỉ độc lập cho từng tiến trình, giúp hạn chế xung đột dữ liệu và tăng tính bảo mật của hệ thống.

Linux Kernel cũng hỗ trợ cơ chế phân trang (Paging) và swapping để tối ưu việc sử dụng RAM. Khi bộ nhớ vật lý không đủ, kernel có thể tạm thời chuyển dữ liệu ít sử dụng sang bộ nhớ lưu trữ thứ cấp nhằm giải phóng RAM cho các tiến trình đang hoạt động. Điều này giúp hệ thống duy trì khả năng hoạt động ổn định ngay cả khi tài nguyên bộ nhớ hạn chế.

Một chức năng quan trọng khác của Linux Kernel là quản lý thiết bị (Device Management). Kernel sử dụng hệ thống Device Driver để giao tiếp với phần cứng như UART, SPI, I2C, Ethernet, GPIO hoặc thiết bị lưu trữ. Device Driver đóng vai trò trung gian giữa phần cứng và hệ điều hành, cho phép ứng dụng người dùng truy cập thiết bị thông qua giao diện chuẩn của Linux.

Linux Kernel hỗ trợ nhiều loại driver khác nhau như Character Device Driver, Block Device Driver và Network Device Driver. Nhờ cơ chế module hóa, các driver có thể được nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại toàn bộ hệ điều hành. Điều này giúp tăng tính linh hoạt và khả năng mở rộng của hệ thống Linux Embedded.

Bên cạnh đó, Linux Kernel còn chịu trách nhiệm quản lý hệ thống tập tin (File System Management). Kernel cung cấp giao diện thống nhất để truy cập dữ liệu lưu trữ trên nhiều loại thiết bị khác nhau như HDD, SSD, eMMC hoặc thẻ nhớ SD.

Linux hỗ trợ nhiều định dạng filesystem như:

- EXT4
- FAT32

- NTFS
- SquashFS
- JFFS2

Kernel quản lý việc đọc, ghi và tổ chức dữ liệu trên thiết bị lưu trữ nhằm đảm bảo tính toàn vẹn và hiệu quả truy xuất dữ liệu. Trong các hệ thống Linux Embedded, việc lựa chọn filesystem phù hợp đóng vai trò quan trọng trong việc tối ưu tốc độ boot, độ bền bộ nhớ flash và hiệu năng hệ thống.

Ngoài các chức năng chính trên, Linux Kernel còn hỗ trợ nhiều cơ chế khác như quản lý mạng, bảo mật hệ thống, đồng bộ tiến trình và giao tiếp liên tiến trình (IPC). Các thành phần này phối hợp với nhau nhằm tạo nên nền tảng hệ điều hành ổn định và có khả năng mở rộng cao.

Trong lĩnh vực hệ thống nhúng và tự động hóa công nghiệp, Linux Kernel được sử dụng rộng rãi nhờ khả năng hỗ trợ đa kiến trúc phần cứng, hiệu năng cao và hệ sinh thái driver phong phú. Đây là nền tảng quan trọng cho nhiều thiết bị embedded hiện đại như gateway công nghiệp, hệ thống IoT, bộ điều khiển tự động hóa và các nền tảng Edge Computing.

3.3.2 User Space và Kernel Space

Trong hệ điều hành Linux, hệ thống được phân chia thành hai không gian hoạt động chính gồm User Space và Kernel Space. Kiến trúc này giúp tăng tính ổn định, bảo mật và khả năng quản lý tài nguyên của hệ điều hành. Việc tách biệt hai không gian cho phép hệ thống kiểm soát quyền truy cập phần cứng cũng như hạn chế các lỗi từ ứng dụng người dùng ảnh hưởng trực tiếp đến kernel.

User Space là không gian dành cho các ứng dụng người dùng hoạt động. Các chương trình như terminal, web browser, application hoặc service thông thường đều được thực thi trong User Space. Các tiến trình trong không gian này chỉ được cấp quyền truy cập giới hạn và không thể thao tác trực tiếp với phần cứng hoặc vùng nhớ hệ thống quan trọng.

Kernel Space là không gian hoạt động của Linux Kernel và các thành phần hệ thống như Device Driver, Memory Manager và Process Scheduler. Các tiến trình trong Kernel Space có quyền truy cập đầy đủ vào tài nguyên phần cứng và bộ nhớ hệ thống. Đây là khu vực có mức đặc quyền cao nhất trong hệ điều hành.

Sự khác biệt giữa User Space và Kernel Space được trình bày trong Bảng 3.4.

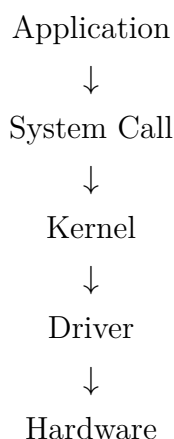
User Space	Kernel Space
Ứng dụng người dùng	Driver và Kernel
Quyền hạn thấp	Quyền hạn cao
Không truy cập trực tiếp phần cứng	Truy cập trực tiếp phần cứng
Được cách ly bởi hệ điều hành	Hoạt động trong vùng đặc quyền

Bảng 3.4: So sánh User Space và Kernel Space

Việc phân tách User Space và Kernel Space giúp hệ điều hành tăng cường tính bảo mật và độ ổn định. Nếu một ứng dụng trong User Space gặp lỗi hoặc bị crash, kernel vẫn có thể tiếp tục hoạt động bình thường mà không làm sập toàn bộ hệ thống. Ngược lại, lỗi xảy ra trong Kernel Space có thể ảnh hưởng trực tiếp đến toàn bộ hệ điều hành do kernel hoạt động với quyền truy cập cao nhất.

Trong Linux, các ứng dụng trong User Space không thể giao tiếp trực tiếp với phần cứng. Thay vào đó, chúng phải sử dụng cơ chế System Call để yêu cầu kernel thực hiện các thao tác cần thiết. System Call đóng vai trò như giao diện trung gian giữa ứng dụng người dùng và Linux Kernel.

Quá trình truy cập phần cứng trong Linux được mô tả như sau:



Khi một ứng dụng cần truy cập thiết bị phần cứng, yêu cầu sẽ được gửi thông qua System Call. Kernel sau đó tiếp nhận yêu cầu và chuyển tiếp đến Device Driver tương ứng. Driver sẽ thực hiện giao tiếp trực tiếp với phần cứng và trả kết quả ngược lại cho kernel trước khi dữ liệu được chuyển về ứng dụng người dùng.

System Call là cơ chế quan trọng giúp kernel kiểm soát toàn bộ quyền truy cập phần cứng trong hệ thống. Điều này giúp ngăn chặn các ứng dụng người dùng thao tác trực tiếp lên tài nguyên hệ thống, từ đó giảm nguy cơ gây lỗi hoặc làm mất ổn định hệ điều hành.

Một số system call phổ biến trong Linux bao gồm:

- `open()` : mở file hoặc thiết bị.

- `read()` : đọc dữ liệu.
- `write()` : ghi dữ liệu.
- `ioctl()` : điều khiển thiết bị.
- `close()` : đóng file hoặc thiết bị.

Trong các hệ thống Linux Embedded, cơ chế User Space và Kernel Space đóng vai trò đặc biệt quan trọng vì nhiều ứng dụng cần giao tiếp với phần cứng ngoại vi như UART, GPIO, SPI hoặc I2C. Device Driver trong Kernel Space chịu trách nhiệm cung cấp giao diện chuẩn để các ứng dụng trong User Space có thể truy cập thiết bị một cách an toàn và hiệu quả.

Ngoài ra, việc tách biệt hai không gian còn giúp tăng khả năng bảo trì và mở rộng hệ thống. Các ứng dụng User Space có thể được cập nhật hoặc thay đổi mà không ảnh hưởng trực tiếp đến kernel. Đồng thời, kernel cũng có thể quản lý tài nguyên hệ thống tập trung nhằm đảm bảo hiệu năng và độ ổn định trong quá trình vận hành.

Trong Linux Embedded hiện đại, kiến trúc User Space và Kernel Space là nền tảng quan trọng cho việc phát triển driver, middleware và application. Cơ chế này giúp Linux trở thành hệ điều hành có tính ổn định cao và phù hợp với các hệ thống nhúng và tự động hóa công nghiệp yêu cầu độ tin cậy lớn.

3.3.3 Linux Device Driver

Trong hệ điều hành Linux, Device Driver là thành phần phần mềm đóng vai trò trung gian giữa Linux Kernel và phần cứng hệ thống. Driver cho phép kernel giao tiếp và điều khiển các thiết bị ngoại vi thông qua giao diện chuẩn của hệ điều hành. Có thể xem Device Driver như một lớp trừu tượng giúp ứng dụng người dùng truy cập phần cứng mà không cần thao tác trực tiếp lên thanh ghi hoặc tín hiệu vật lý của thiết bị.

Trong Linux, mỗi thiết bị phần cứng thường được biểu diễn dưới dạng file thiết bị trong thư mục `/dev`. Khi ứng dụng người dùng thực hiện thao tác đọc hoặc ghi dữ liệu lên file thiết bị, yêu cầu sẽ được chuyển đến driver tương ứng trong kernel space để xử lý. Cơ chế này giúp chuẩn hóa phương thức giao tiếp với phần cứng và tăng tính độc lập giữa application và hardware platform.

Linux Device Driver chịu trách nhiệm thực hiện nhiều chức năng quan trọng như:

- Khởi tạo thiết bị phần cứng.
- Cấu hình thanh ghi điều khiển.
- Truyền và nhận dữ liệu.

- Quản lý interrupt.
- Đồng bộ truy cập thiết bị.
- Cung cấp giao diện cho user space.

Thông qua driver, Linux Kernel có thể quản lý nhiều loại thiết bị phần cứng khác nhau mà không cần ứng dụng người dùng hiểu chi tiết cấu trúc bên trong của thiết bị. Điều này giúp tăng tính linh hoạt và khả năng mở rộng của hệ điều hành.

Trong Linux, Device Driver thường được chia thành ba nhóm chính gồm Character Device Driver, Block Device Driver và Network Device Driver. Mỗi loại driver được thiết kế để phục vụ một kiểu giao tiếp phần cứng khác nhau.

Các loại Linux Device Driver phổ biến được trình bày trong Bảng 3.5.

Loại Driver	Ví dụ
Character Device	UART
Block Device	SSD
Network Device	Ethernet

Bảng 3.5: Các loại Linux Device Driver

Character Device Driver là loại driver cho phép truyền dữ liệu theo từng byte hoặc từng ký tự liên tiếp. Các thiết bị như UART, GPIO, I2C hoặc SPI thường sử dụng mô hình character device. Trong Linux, Character Device Driver thường hỗ trợ các system call như `open()`, `read()`, `write()` và `ioctl()` nhằm cho phép ứng dụng user space giao tiếp với phần cứng.

Đặc điểm của Character Device là dữ liệu được xử lý tuần tự và không hỗ trợ truy cập ngẫu nhiên theo block. Đây là loại driver phổ biến trong các hệ thống Linux Embedded do nhiều peripheral hoạt động theo cơ chế truyền dữ liệu nối tiếp.

Block Device Driver là loại driver được sử dụng cho các thiết bị lưu trữ như HDD, SSD hoặc thẻ nhớ SD. Khác với Character Device, Block Device hỗ trợ truy cập dữ liệu theo block cố định và cho phép truy cập ngẫu nhiên đến vị trí dữ liệu trên thiết bị lưu trữ.

Linux Kernel sử dụng Block Device Driver kết hợp với filesystem nhằm quản lý dữ liệu trên thiết bị lưu trữ một cách hiệu quả. Các block device thường được tối ưu cho tốc độ truy xuất dữ liệu và khả năng quản lý bộ nhớ đệm (buffer cache).

Network Device Driver là loại driver phục vụ giao tiếp mạng trong Linux. Các thiết bị như Ethernet Controller hoặc Wireless Adapter sử dụng loại driver này để truyền và nhận packet dữ liệu thông qua network stack của kernel.

Network Driver có nhiệm vụ xử lý packet, quản lý buffer truyền nhận dữ liệu và phối hợp với giao thức mạng trong Linux Kernel như TCP/IP hoặc UDP. Đây là thành phần

quan trọng trong các hệ thống nhúng có chức năng kết nối mạng hoặc truyền thông công nghiệp.

Trong Linux Embedded, Device Driver thường được phát triển dưới dạng Kernel Module nhằm tăng tính linh hoạt cho hệ thống. Kernel Module có thể được nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại toàn bộ Linux Kernel. Điều này giúp quá trình phát triển và bảo trì driver trở nên thuận tiện hơn.

Ngoài ra, Linux còn hỗ trợ cơ chế Device Tree nhằm mô tả phần cứng và liên kết driver với thiết bị tương ứng trong hệ thống nhúng. Khi kernel khởi động, Device Tree sẽ cung cấp thông tin phần cứng để kernel tự động khởi tạo và nạp driver phù hợp.

3.3.4 Device Tree

Trong các hệ thống Linux Embedded hiện đại, Device Tree là cơ chế được sử dụng để mô tả cấu trúc phần cứng của hệ thống một cách độc lập với Linux Kernel. Device Tree cho phép kernel nhận biết và cấu hình các thiết bị phần cứng mà không cần chỉnh sửa trực tiếp mã nguồn kernel. Đây là thành phần quan trọng trong các nền tảng nhúng sử dụng kiến trúc ARM, RISC-V và nhiều hệ thống SoC hiện đại khác.

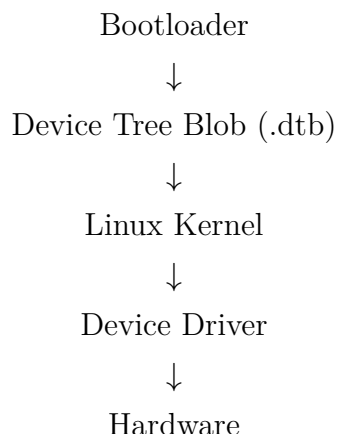
Trước khi Device Tree được sử dụng rộng rãi, thông tin phần cứng thường được hard-code trực tiếp trong Linux Kernel. Cách tiếp cận này làm tăng độ phức tạp của kernel và gây khó khăn trong việc hỗ trợ nhiều nền tảng phần cứng khác nhau. Khi mỗi board phần cứng yêu cầu chỉnh sửa mã nguồn kernel riêng biệt, quá trình bảo trì và nâng cấp hệ thống trở nên phức tạp và khó mở rộng.

Để giải quyết vấn đề này, Linux giới thiệu cơ chế Device Tree nhằm tách biệt thông tin phần cứng ra khỏi kernel source code. Nhờ đó, cùng một Linux Kernel có thể hoạt động trên nhiều nền tảng phần cứng khác nhau chỉ bằng cách thay đổi Device Tree tương ứng.

Device Tree mô tả phần cứng dưới dạng cấu trúc cây (Tree Structure). Mỗi node trong cây đại diện cho một thành phần phần cứng như CPU, bộ nhớ, UART, SPI, I2C, GPIO hoặc các thiết bị ngoại vi khác. Các node này chứa thông tin cấu hình cần thiết để kernel và driver có thể giao tiếp với phần cứng.

Thông thường, Device Tree được viết dưới dạng file `.dts` (Device Tree Source). Sau đó, file này được biên dịch thành file nhị phân `.dtb` (Device Tree Blob) để kernel sử dụng trong quá trình boot.

Quá trình hoạt động của Device Tree được mô tả như sau:



Trong quá trình khởi động hệ thống, bootloader sẽ nạp Linux Kernel cùng với file Device Tree Blob vào bộ nhớ. Kernel sau đó phân tích Device Tree để xác định cấu hình phần cứng của hệ thống và tự động khởi tạo các driver tương ứng.

Một Device Tree thường bao gồm các thông tin như:

- Kiến trúc CPU.
- Dung lượng bộ nhớ RAM.
- Địa chỉ thanh ghi phần cứng.
- Cấu hình GPIO.
- Thông số UART, SPI, I2C.
- Interrupt Request (IRQ).
- Clock và Power Management.

Ví dụ cấu hình UART trong Device Tree:

```
uart0 {  
    status = "okay";  
    current-speed = <115200>;  
};
```

Trong ví dụ trên, node `uart0` đại diện cho bộ điều khiển UART của hệ thống. Thuộc tính `status = "okay"` cho biết thiết bị được kích hoạt, trong khi `current-speed` xác định tốc độ baudrate sử dụng cho giao tiếp UART.

Một ưu điểm quan trọng của Device Tree là khả năng tách biệt phần cứng và phần mềm. Linux Kernel và driver không cần phụ thuộc trực tiếp vào cấu trúc phần cứng cụ

thể của từng board. Điều này giúp tăng khả năng tái sử dụng kernel và giảm khối lượng mã nguồn cần bảo trì.

Ngoài ra, Device Tree còn hỗ trợ khả năng mở rộng hệ thống nhúng. Khi cần bổ sung thiết bị phần cứng mới, người phát triển chỉ cần cập nhật Device Tree mà không cần chỉnh sửa sâu vào Linux Kernel. Điều này giúp rút ngắn thời gian phát triển và tăng tính linh hoạt trong quá trình tích hợp phần cứng.

Trong Linux Embedded, Device Tree thường được sử dụng kết hợp với Device Driver thông qua cơ chế compatible string. Mỗi node phần cứng sẽ khai báo thuộc tính `compatible` để kernel xác định driver phù hợp cho thiết bị.

Ví dụ:

```
compatible = "vendor,uart-device";
```

Khi kernel khởi động, hệ thống driver model của Linux sẽ tìm driver có compatible string tương ứng và tự động bind driver với thiết bị phần cứng.

Device Tree đặc biệt quan trọng trong các hệ thống SoC hiện đại vì số lượng peripheral và cấu hình phần cứng thường rất lớn. Việc mô tả phần cứng bằng Device Tree giúp Linux Kernel duy trì kiến trúc gọn gàng, tăng khả năng hỗ trợ đa nền tảng và đơn giản hóa quá trình phát triển hệ thống nhúng.

Trong Yocto Project, Device Tree thường được build cùng với Linux Kernel và được tích hợp trực tiếp vào Embedded Linux Image. Nhờ đó, hệ thống Linux nhúng có thể khởi động và nhận diện phần cứng một cách tự động trên nền tảng target device.

3.3.5 Kernel Module

Trong Linux Kernel, Kernel Module là thành phần phần mềm có thể được nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại hoặc khởi động lại toàn bộ hệ điều hành. Cơ chế này giúp Linux tăng tính linh hoạt trong việc mở rộng chức năng hệ thống, đặc biệt trong các môi trường Linux Embedded và hệ thống nhúng công nghiệp.

Kernel Module thường được sử dụng để triển khai các thành phần như Device Driver, File System, Network Protocol hoặc các chức năng mở rộng của kernel. Thay vì tích hợp toàn bộ chức năng trực tiếp vào Linux Kernel, các module có thể được quản lý độc lập và chỉ nạp vào hệ thống khi cần thiết. Điều này giúp giảm kích thước kernel và tối ưu tài nguyên hệ thống.

Trong Linux, kernel module sau khi biên dịch thường có phần mở rộng `.ko` (Kernel Object). File module chứa mã máy có thể được Linux Kernel nạp động vào kernel space trong quá trình runtime.

Một ưu điểm quan trọng của Kernel Module là khả năng phát triển và kiểm thử driver mà không cần build lại toàn bộ Linux Kernel. Điều này giúp rút ngắn đáng kể thời gian phát triển hệ thống nhúng, đặc biệt đối với các dự án cần thường xuyên cập nhật hoặc tùy chỉnh driver phần cứng.

Ngoài ra, cơ chế module hóa còn giúp tăng khả năng bảo trì và mở rộng hệ thống. Các chức năng mới có thể được bổ sung thông qua việc nạp module tương ứng mà không ảnh hưởng đến các thành phần khác của kernel.

Linux cung cấp nhiều công cụ để quản lý kernel module trong hệ thống. Các lệnh phổ biến được trình bày như sau:

```
insmod  
rmmod  
modprobe  
lsmod
```

Lệnh `insmod` được sử dụng để nạp kernel module vào Linux Kernel. Khi module được nạp thành công, kernel sẽ thực thi các hàm khởi tạo và đăng ký chức năng của module với hệ thống.

Lệnh `rmmod` được sử dụng để gỡ module khỏi kernel. Khi module bị gỡ bỏ, kernel sẽ giải phóng tài nguyên liên quan và thực thi các hàm cleanup của module.

Lệnh `modprobe` là phiên bản nâng cao của `insmod`. Công cụ này hỗ trợ tự động kiểm tra và nạp các dependency cần thiết trước khi module chính được load vào hệ thống.

Lệnh `lsmod` được sử dụng để hiển thị danh sách các kernel module đang hoạt động trong hệ thống Linux. Thông tin hiển thị thường bao gồm tên module, dung lượng bộ nhớ sử dụng và số lượng tiến trình đang sử dụng module đó.

Trong Linux Kernel, mỗi module thường bao gồm hai thành phần chính là hàm khởi tạo và hàm giải phóng tài nguyên. Cấu trúc cơ bản của kernel module được mô tả như sau:

```
module_init()  
module_exit()
```

Trong đó:

- `module_init()` được gọi khi nạp module.
- `module_exit()` được gọi khi gỡ module.

Hàm `module_init()` được sử dụng để thực hiện các thao tác khởi tạo khi module được load vào kernel. Các công việc phổ biến trong hàm này bao gồm đăng ký device driver, cấp phát bộ nhớ, khởi tạo hardware hoặc thiết lập interrupt handler.

Ngược lại, hàm `module_exit()` chịu trách nhiệm giải phóng tài nguyên khi module bị gỡ khỏi hệ thống. Điều này bao gồm giải phóng bộ nhớ, unregister driver hoặc đóng các kết nối phần cứng liên quan.

Một kernel module thông thường còn bao gồm nhiều thành phần metadata khác nhằm cung cấp thông tin cho Linux Kernel như:

- Tên tác giả module.
- License sử dụng.
- Phiên bản module.
- Mô tả chức năng module.

Ví dụ:

```
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("Author Name");  
MODULE_DESCRIPTION("Linux Kernel Module");
```

Các metadata này giúp kernel quản lý module và kiểm tra tính tương thích giữa module với phiên bản Linux Kernel đang sử dụng.

Trong Linux Embedded, kernel module thường được sử dụng để triển khai Device Driver cho các peripheral như UART, GPIO, SPI, I2C hoặc Ethernet. Việc phát triển driver dưới dạng module giúp hệ thống linh hoạt hơn trong quá trình debug và nâng cấp phần mềm.

Ngoài ra, Linux Kernel còn hỗ trợ cơ chế tự động nạp module thông qua `udev` hoặc `modprobe`. Khi hệ thống phát hiện phần cứng tương ứng, kernel sẽ tự động load driver cần thiết mà không yêu cầu thao tác thủ công từ người dùng.

Trong các hệ thống nhúng và tự động hóa công nghiệp, Kernel Module đóng vai trò quan trọng trong việc xây dựng kiến trúc phần mềm linh hoạt, dễ bảo trì và có khả năng mở rộng cao. Đây là nền tảng quan trọng cho việc phát triển driver và các chức năng hệ thống trong Linux Embedded hiện đại.

3.3.6 Kernel Logging

Trong Linux Kernel, cơ chế logging được sử dụng để ghi nhận thông tin hoạt động của hệ thống trong quá trình runtime. Kernel Logging đóng vai trò quan trọng trong việc theo dõi trạng thái hệ thống, kiểm tra hoạt động của driver và hỗ trợ debug trong quá trình phát triển Linux Embedded.

Khác với các ứng dụng trong User Space có thể sử dụng các hàm như `printf()` để xuất dữ liệu ra terminal, Linux Kernel hoạt động trong kernel space nên không thể sử dụng trực tiếp các hàm thư viện chuẩn của user space. Thay vào đó, kernel sử dụng hàm `printk()` để ghi log vào kernel log buffer.

Hàm `printk()` là cơ chế logging chuẩn của Linux Kernel và được sử dụng rộng rãi trong quá trình phát triển kernel module, device driver và các thành phần hệ thống mức thấp.

Khi hàm `printk()` được thực thi, thông điệp sẽ được ghi vào vùng nhớ kernel log buffer. Các thông tin này sau đó có thể được truy xuất để kiểm tra trạng thái hoạt động của hệ thống hoặc hỗ trợ quá trình debug.

Trong Linux, thông điệp kernel thường được kiểm tra bằng lệnh:

`dmesg`

Lệnh `dmesg` cho phép hiển thị toàn bộ nội dung trong kernel ring buffer, bao gồm các thông tin liên quan đến quá trình boot hệ thống, hoạt động của driver, lỗi phần cứng và các thông báo runtime từ Linux Kernel.

Kernel logging đóng vai trò đặc biệt quan trọng trong Linux Embedded vì nhiều thành phần hệ thống như Device Driver hoặc Kernel Module hoạt động trực tiếp với phần cứng và không có giao diện debug trực quan như các ứng dụng user space. Do đó, `printk()` thường được sử dụng để theo dõi trạng thái hoạt động của driver và kiểm tra luồng thực thi trong kernel.

Ngoài chức năng debug, kernel logging còn hỗ trợ giám sát lỗi hệ thống và phát hiện các sự cố liên quan đến phần cứng hoặc tài nguyên hệ thống. Các thông điệp log có thể cung cấp thông tin về lỗi memory access, kernel panic, interrupt hoặc trạng thái của peripheral trong quá trình hoạt động.

Linux Kernel hỗ trợ nhiều mức độ log khác nhau nhằm phân loại mức độ quan trọng của thông điệp. Các mức log phổ biến bao gồm:

- `KERN_EMERG`: lỗi nghiêm trọng, hệ thống không thể hoạt động.
- `KERN_ALERT`: yêu cầu xử lý ngay lập tức.
- `KERN_ERR`: thông báo lỗi hệ thống.
- `KERN_WARNING`: cảnh báo hệ thống.
- `KERN_INFO`: thông tin hoạt động thông thường.
- `KERN_DEBUG`: thông tin phục vụ debug.

Ví dụ:

```
printk(KERN_INFO "Driver Initialized");
```

Trong ví dụ trên, thông điệp được gắn mức log `KERN_INFO`, cho biết đây là thông tin hoạt động thông thường của hệ thống.

Linux Kernel sử dụng cơ chế ring buffer để lưu trữ log. Khi vùng nhớ log đầy, các thông điệp cũ sẽ bị ghi đè bởi thông điệp mới. Cơ chế này giúp kernel quản lý bộ nhớ logging hiệu quả và tránh tiêu tốn quá nhiều tài nguyên hệ thống.

Trong các hệ thống Linux Embedded và tự động hóa công nghiệp, kernel logging là công cụ quan trọng phục vụ quá trình phát triển và bảo trì hệ thống. Thông qua các thông điệp log, người phát triển có thể kiểm tra hoạt động của driver, theo dõi trạng thái thiết bị ngoại vi và nhanh chóng xác định nguyên nhân gây lỗi trong quá trình vận hành.

Ngoài ra, kernel logging còn hỗ trợ việc phân tích hiệu năng và đánh giá độ ổn định của hệ thống nhúng trong các môi trường công nghiệp yêu cầu độ tin cậy cao. Đây là một trong những cơ chế debug cơ bản nhưng rất quan trọng trong phát triển Linux Kernel và Device Driver.

3.3.7 Kernel Build System

Trong Linux Kernel, việc biên dịch kernel module được thực hiện thông qua Kernel Build System, thường được gọi là Kbuild. Đây là hệ thống build chính thức của Linux Kernel, chịu trách nhiệm quản lý quá trình biên dịch kernel, device driver và các kernel module.

Kbuild được thiết kế nhằm hỗ trợ quá trình build một cách thống nhất và tự động hóa trên nhiều kiến trúc phần cứng khác nhau. Hệ thống này cho phép các module bên ngoài (External Module) có thể được biên dịch tương thích với Linux Kernel mà không cần chỉnh sửa trực tiếp mã nguồn kernel.

Trong quá trình phát triển Linux Embedded, Kernel Build System đóng vai trò rất quan trọng vì phần lớn device driver thường được xây dựng dưới dạng kernel module độc lập. Việc sử dụng Kbuild giúp đảm bảo module được biên dịch với đúng compiler, header file và cấu hình kernel tương ứng với hệ thống đích.

Linux Kernel sử dụng Makefile để mô tả quá trình build module. Một Makefile cơ bản cho kernel module thường bao gồm khai báo object module như sau:

```
obj-m := my_driver.o
```

Trong đó:

- `obj-m` là biến được Kbuild sử dụng để khai báo external kernel module.
- `my_driver.o` là object file được tạo ra từ source code của module.

Khi quá trình build hoàn tất, Linux Kernel sẽ tạo file kernel object có phần mở rộng `.ko` để có thể nạp động vào kernel trong runtime.

Một trong những cơ chế quan trọng của Kbuild là khả năng sử dụng Linux Kernel Source Tree để build external module. Điều này được thực hiện thông qua lệnh:

```
$(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules
```

Lệnh trên cho phép Makefile gọi hệ thống build của Linux Kernel để thực hiện biên dịch module bên ngoài.

Trong đó:

- `$(MAKE)` gọi chương trình make.
- `-C $(KERNEL_SRC)` chuyển thư mục làm việc sang kernel source tree.
- `M=$(SRC)` chỉ định thư mục chứa source code của external module.
- `modules` yêu cầu kernel build system thực hiện build module.

Thông qua cơ chế này, external module có thể sử dụng toàn bộ môi trường build của Linux Kernel như:

- Compiler configuration.
- Kernel header.
- Symbol table.
- Cross-compilation toolchain.
- Kernel configuration.

Điều này giúp đảm bảo module được build tương thích hoàn toàn với phiên bản Linux Kernel đang sử dụng.

Trong Linux Embedded, Kbuild còn hỗ trợ cơ chế cross-compilation nhằm biên dịch kernel module cho các nền tảng phần cứng khác nhau như ARM, ARM64 hoặc RISC-V. BitBake và Yocto Project thường tích hợp trực tiếp với Kernel Build System để tự động hóa quá trình build external module trong Embedded Linux Image.

Ngoài việc build module, Kbuild còn hỗ trợ quản lý dependency giữa các source file và tự động tạo các file trung gian cần thiết trong quá trình biên dịch. Điều này giúp giảm đáng kể độ phức tạp khi phát triển các driver hoặc module có cấu trúc lớn.

Một ưu điểm quan trọng của Kernel Build System là khả năng đồng bộ với cấu hình kernel hiện tại. Khi Linux Kernel được cấu hình lại, các module build bằng Kbuild sẽ tự động sử dụng đúng configuration và symbol tương ứng, giúp tránh lỗi không tương thích trong quá trình runtime.

Bên cạnh đó, Kbuild còn hỗ trợ cơ chế incremental build nhằm chỉ biên dịch lại các thành phần bị thay đổi thay vì build lại toàn bộ module. Điều này giúp tối ưu thời gian phát triển và tăng hiệu quả trong quá trình debug driver.

Trong các hệ thống Linux Embedded và tự động hóa công nghiệp, Kernel Build System là nền tảng quan trọng cho việc phát triển và tích hợp Device Driver. Nhờ khả năng hỗ trợ build module linh hoạt, tương thích đa nền tảng và tích hợp chặt chẽ với Linux Kernel, Kbuild đã trở thành thành phần không thể thiếu trong hệ sinh thái phát triển Linux Embedded hiện đại.

3.3.8 Cơ chế tự động nạp module

Trong Linux, kernel module có thể được nạp thủ công bằng các lệnh như `insmod` hoặc `modprobe`. Tuy nhiên, trong các hệ thống Linux Embedded và tự động hóa công nghiệp, nhiều driver cần được khởi tạo tự động ngay khi hệ thống khởi động nhằm đảm bảo các thiết bị phần cứng sẵn sàng hoạt động mà không yêu cầu thao tác từ người dùng.

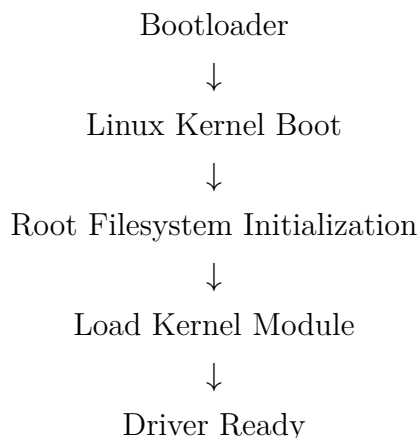
Để hỗ trợ cơ chế này, Yocto Project cung cấp khả năng tự động nạp kernel module thông qua metadata trong recipe. Khi image Linux được build, Yocto sẽ tự động tạo cấu hình để kernel nạp module tương ứng trong quá trình boot hệ thống.

Cơ chế tự động nạp module trong Yocto thường được khai báo bằng biến:

```
KERNEL_MODULE_AUTOLOAD += "my_driver"
```

Biến `KERNEL_MODULE_AUTOLOAD` được sử dụng để chỉ định danh sách kernel module cần tự động load khi hệ thống khởi động. Khi module được khai báo trong biến này, Yocto sẽ tạo các cấu hình cần thiết trong root filesystem nhằm đảm bảo module được kernel nạp tự động trong runtime.

Cơ chế hoạt động của quá trình tự động nạp module có thể được mô tả như sau:



Trong quá trình khởi động hệ thống, Linux Kernel trước tiên sẽ khởi tạo các thành phần cơ bản như memory manager, scheduler và filesystem. Sau khi root filesystem được mount hoàn tất, hệ thống init sẽ thực hiện quá trình load các kernel module được cấu hình tự động.

Các module thường được nạp thông qua công cụ `modprobe`. Công cụ này không chỉ load module chính mà còn tự động kiểm tra và nạp các dependency cần thiết trước khi module hoạt động. Điều này giúp đảm bảo driver được khởi tạo đầy đủ và ổn định trong hệ thống.

Trong Linux Embedded, cơ chế tự động nạp module đặc biệt quan trọng đối với các driver phần cứng như:

- UART Driver.
- SPI Driver.
- I2C Driver.
- Ethernet Driver.
- GPIO Driver.
- USB Driver.

Nếu các driver này không được load đúng thời điểm trong quá trình boot, hệ thống có thể không nhận diện được peripheral hoặc không thể giao tiếp với phần cứng.

Một ưu điểm quan trọng của cơ chế tự động nạp module là tăng tính tự động hóa và giảm thao tác cấu hình thủ công sau khi triển khai hệ thống. Điều này đặc biệt cần thiết trong các hệ thống nhúng công nghiệp hoặc thiết bị IoT, nơi hệ thống cần khởi động và hoạt động ổn định mà không có sự can thiệp của người dùng.

Ngoài ra, việc quản lý module thông qua metadata của Yocto giúp đảm bảo tính tái lập của hệ thống build. Khi recipe được sử dụng lại trên nhiều môi trường hoặc nhiều

thiết bị khác nhau, cấu hình auto-load module vẫn được giữ nguyên và hoạt động đồng nhất.

Trong Linux Kernel, các module được tự động load thường được lưu tại thư mục:

```
/lib/modules/<kernel-version>/
```

Hệ thống sẽ sử dụng các file cấu hình module để xác định module cần nạp trong quá trình boot. Yocto tự động tạo các cấu hình này dựa trên biến `KERNEL_MODULE_AUTOLOAD` trong recipe của package.

Ngoài phương pháp cấu hình trong Yocto, Linux còn hỗ trợ nhiều cơ chế tự động nạp module khác như:

- Udev.
- Systemd Module Loader.
- Init Script.

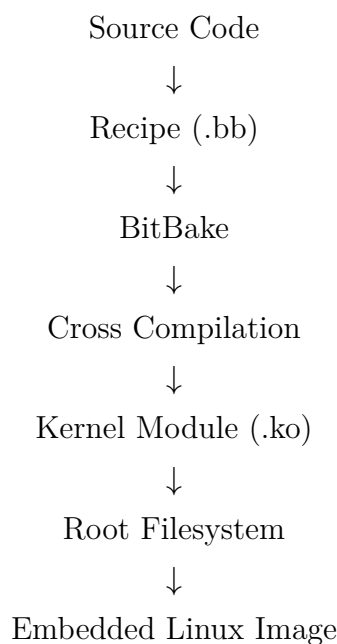
Tuy nhiên, trong các hệ thống Linux Embedded sử dụng Yocto Project, việc quản lý auto-load module thông qua recipe được xem là phương pháp tối ưu vì toàn bộ cấu hình được tích hợp trực tiếp vào pipeline build của hệ thống.

Cơ chế tự động nạp module giúp đảm bảo driver phần cứng luôn sẵn sàng sau khi hệ thống khởi động. Điều này góp phần tăng độ ổn định, giảm thời gian cấu hình và nâng cao khả năng triển khai thực tế của hệ thống Linux Embedded.

3.3.9 Xây dựng và tích hợp Driver trong Yocto

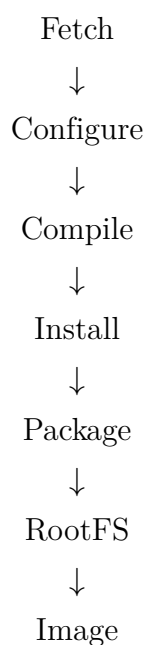
Trong Yocto Project, quá trình xây dựng hệ thống Linux Embedded được thực hiện theo mô hình pipeline tự động hóa thông qua BitBake và metadata của recipe. Toàn bộ quy trình từ mã nguồn driver đến image Linux hoàn chỉnh đều được quản lý thống nhất nhằm đảm bảo tính tái lập và khả năng mở rộng của hệ thống.

Quy trình build tổng quát trong Yocto được mô tả như sau:



Trong quy trình này, source code và metadata của driver được định nghĩa thông qua recipe BitBake. BitBake sau đó phân tích dependency và thực hiện pipeline build bao gồm các giai đoạn fetch, configure, compile, install và package nhằm tạo ra kernel module tương ứng.

Pipeline build của Yocto có thể được mô tả như sau:



Trong đó, BitBake sẽ tự động tải source code, cấu hình môi trường cross-compilation, biên dịch kernel module và tích hợp package vào root filesystem của Embedded Linux Image.

Để BitBake có thể nhận diện recipe và metadata của driver, layer tùy chỉnh cần

được tích hợp vào môi trường build bằng lệnh:

```
bitbake-layers add-layer ../meta-my-driver
```

Sau khi layer được thêm vào hệ thống, các recipe trong layer sẽ được BitBake phân tích và đưa vào dependency graph của quá trình build.

Driver được tích hợp vào Embedded Linux Image thông qua biến:

```
IMAGE_INSTALL:append = " my-driver"
```

Biến này cho phép package của driver được tự động đưa vào root filesystem trong quá trình build image. Nhờ đó, hệ thống Linux Embedded sau khi boot sẽ có đầy đủ module và các thành phần cần thiết phục vụ quá trình runtime.

Sau khi hệ thống khởi động, kernel module có thể được kiểm tra thông qua các lệnh quản lý module của Linux. Một số lệnh phổ biến được trình bày trong Bảng 3.6.

Lệnh	Chức năng
lsmod	Kiểm tra module đang nạp
dmesg	Kiểm tra kernel log
modinfo	Xem thông tin module

Bảng 3.6: Các lệnh kiểm tra Kernel Module

Lệnh `lsmod` cho phép kiểm tra danh sách kernel module đang hoạt động trong hệ thống. Lệnh `dmesg` được sử dụng để xem thông điệp kernel log nhằm xác nhận trạng thái hoạt động của driver. Trong khi đó, `modinfo` cung cấp thông tin metadata của kernel module như phiên bản, license và vị trí file module trong filesystem.

Nhờ cơ chế build tự động và quản lý metadata của Yocto Project, quá trình xây dựng và tích hợp driver vào hệ thống Linux Embedded được thực hiện một cách đồng bộ, linh hoạt và có khả năng tái sử dụng cao. Điều này giúp giảm độ phức tạp trong phát triển hệ thống nhúng và tăng hiệu quả triển khai trong các ứng dụng tự động hóa công nghiệp.

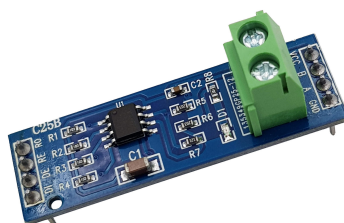
3.4 Bảo mật hệ điều hành Linux nhúng

3.5 Edge AI trong hệ thống nhúng

3.6 Giao diện người dùng trên thiết bị nhúng

3.7 Giao tiếp phân cứng công nghiệp

3.7.1 Chuẩn vật lý RS485



Hình 3.4: RS485 TTL

Chuẩn RS485 (Recommended Standard 485) là một chuẩn truyền thông ở tầng vật lý, được Hiệp hội Công nghiệp Điện tử (EIA) ban hành nhằm phục vụ các hệ thống truyền dữ liệu nối tiếp trong môi trường công nghiệp. RS485 được thiết kế để đáp ứng yêu cầu truyền dữ liệu ổn định trên khoảng cách xa, chống nhiễu tốt và hỗ trợ nhiều thiết bị cùng kết nối trên một đường truyền, do đó được sử dụng rộng rãi trong các hệ thống điều khiển, giám sát và tự động hóa công nghiệp.

Về mặt nguyên lý hoạt động, RS485 sử dụng phương pháp truyền tín hiệu vi sai (differential signaling) thông qua hai dây tín hiệu A và tín hiệu B. Dữ liệu được biểu diễn bằng sự chênh lệch điện áp giữa hai dây này thay vì so với mass (hoặc GND) như các chuẩn truyền đơn cực. Cách truyền vi sai giúp RS485 có khả năng chống nhiễu điện từ cao, đặc biệt phù hợp với môi trường công nghiệp nơi tồn tại nhiều nguồn nhiễu như động cơ, biến tần và thiết bị công suất lớn.

Chuẩn RS485 hỗ trợ cấu trúc mạng đa điểm (multi-drop), cho phép nhiều thiết bị cùng kết nối trên một bus truyền thông duy nhất. Về lý thuyết, một mạng RS485 có thể kết nối tối đa 32 thiết bị tiêu chuẩn (1 unit load), và con số này có thể tăng lên khi sử dụng các bộ thu phát (transceiver) hiện đại với tải nhỏ hơn. Khoảng cách truyền tối đa của RS485 có thể đạt tới 1200 m trong điều kiện tốc độ thấp, trong khi tốc độ

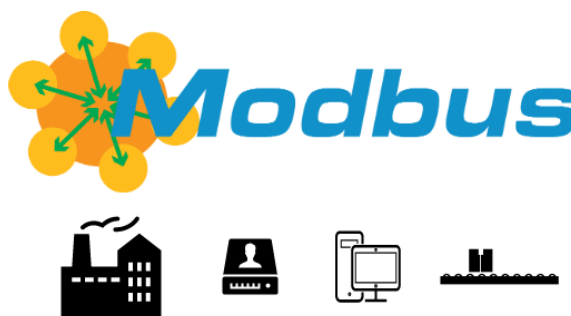
truyền dữ liệu có thể lên tới hàng Mbps khi khoảng cách ngắn, cho thấy sự linh hoạt trong thiết kế hệ thống.

RS485 không quy định giao thức truyền thông ở tầng trên mà chỉ định nghĩa các đặc tính điện và tín hiệu ở tầng vật lý. Do đó, RS485 thường được kết hợp với các giao thức truyền thông công nghiệp như Modbus RTU hoặc giao thức tùy biến để hình thành hệ thống truyền thông hoàn chỉnh. Trong thực tế, Modbus RTU trên nền RS485 là một trong những sự kết hợp phổ biến nhất trong các hệ thống giám sát và điều khiển công nghiệp.

Về cấu hình truyền thông, RS485 có thể hoạt động ở chế độ bán song công (half-duplex) hoặc song công toàn phần (full-duplex), tuy nhiên bán song công là chế độ được sử dụng phổ biến nhất nhằm giảm số lượng dây dẫn và chi phí triển khai. Để đảm bảo tín hiệu ổn định, mạng RS485 yêu cầu sử dụng điện trở kết thúc (termination resistor) ở hai đầu bus truyền nhằm giảm phản xạ tín hiệu, đồng thời có thể cần điện trở bias để xác định trạng thái đường truyền khi không có thiết bị phát.

Nhờ các đặc tính nổi bật như khả năng truyền xa, chống nhiễu tốt, hỗ trợ nhiều thiết bị và chi phí triển khai thấp, chuẩn RS485 vẫn giữ vai trò quan trọng trong các hệ thống công nghiệp hiện đại. Mặc dù đã xuất hiện nhiều chuẩn truyền thông mới, RS485 vẫn là lựa chọn ưu tiên trong các ứng dụng yêu cầu độ tin cậy cao, cấu trúc đơn giản và khả năng hoạt động ổn định trong môi trường khắc nghiệt.

3.7.2 Giao thức Modbus RTU và cấu trúc khung tin



Hình 3.5: Modbus RTU

a. Giao thức Modbus RTU

Modbus RTU (Remote Terminal Unit) là một giao thức truyền thông công nghiệp phổ biến, được phát triển bởi Modicon vào năm 1979. Đây là giao thức dạng master-slave (hiện nay còn gọi là client-server), được thiết kế nhằm cho phép các thiết bị điều khiển và giám sát trong công nghiệp trao đổi dữ liệu với nhau một cách đơn giản, tin cậy và hiệu quả. Nhờ tính mở, dễ triển khai và khả năng tương thích cao, Modbus RTU hiện

vẫn được sử dụng rộng rãi trong các hệ thống SCADA, PLC, cảm biến và thiết bị đo lường công nghiệp.

Modbus RTU thường được triển khai trên các chuẩn truyền thông vật lý nối tiếp như RS232 và đặc biệt phổ biến trên RS485 mà nhóm đang thực hiện trong dự án, do RS485 hỗ trợ truyền xa, chống nhiễu tốt và cho phép nhiều thiết bị cùng kết nối trên một đường bus. Giao thức Modbus RTU không phụ thuộc vào phần cứng cụ thể mà chỉ quy định cách thức tổ chức dữ liệu và trao đổi thông tin giữa các thiết bị.

Trong mô hình hoạt động của Modbus RTU, thiết bị master đóng vai trò khởi tạo và điều khiển toàn bộ quá trình truyền thông, trong khi các thiết bị slave chỉ phản hồi khi nhận được yêu cầu hợp lệ từ master. Mỗi thiết bị slave được gán một địa chỉ duy nhất trên bus, giúp master xác định đúng thiết bị cần giao tiếp. Cơ chế này giúp tránh xung đột dữ liệu và đảm bảo tính tuần tự trong quá trình truyền thông.

b. Cấu trúc khung tin

Modbus RTU truyền dữ liệu dưới dạng các khung tin (frame) được mã hóa theo dạng nhị phân, cho phép tiết kiệm băng thông và nâng cao hiệu quả truyền thông. Mỗi khung tin Modbus RTU bao gồm các trường dữ liệu được sắp xếp theo thứ tự xác định, đảm bảo thiết bị nhận có thể giải mã chính xác nội dung thông tin.

Một khung tin Modbus RTU tiêu chuẩn bao gồm các thành phần chính: địa chỉ thiết bị, mã hàm, vùng dữ liệu và mã kiểm tra lỗi CRC. Khung tin được phân tách với các khung khác bằng một khoảng im lặng trên đường truyền (silent interval), có độ dài tối thiểu tương đương 3.5 ký tự, giúp thiết bị nhận xác định ranh giới giữa các khung dữ liệu.

Trường địa chỉ thiết bị có độ dài 1 byte, dùng để xác định slave mà master muốn giao tiếp. Địa chỉ hợp lệ nằm trong khoảng từ 1 đến 247, trong khi địa chỉ 0 được sử dụng cho chế độ broadcast, cho phép master gửi lệnh đến tất cả các slave mà không yêu cầu phản hồi.

Mã hàm (Function Code), cũng có độ dài 1 byte, dùng để chỉ định loại thao tác mà master yêu cầu thực hiện. Các mã hàm phổ biến bao gồm đọc thanh ghi (Read Holding Registers), ghi thanh ghi (Write Single Register), đọc coil hoặc input status. Dựa vào mã hàm, thiết bị slave sẽ biết cách xử lý yêu cầu và chuẩn bị dữ liệu phản hồi phù hợp.

Phần dữ liệu có độ dài thay đổi tùy thuộc vào mã hàm được sử dụng. Trường này có thể chứa địa chỉ thanh ghi, số lượng thanh ghi cần đọc/ghi hoặc dữ liệu cụ thể cần truyền. Cấu trúc linh hoạt của vùng dữ liệu cho phép Modbus RTU thích ứng với nhiều loại thiết bị và nhu cầu truyền thông khác nhau trong công nghiệp.

Mỗi khung tin Modbus RTU kết thúc bằng trường CRC (Cyclic Redundancy Check) dài 2 byte. CRC được sử dụng để kiểm tra lỗi trong quá trình truyền dữ liệu, giúp phát hiện các lỗi do nhiễu hoặc sai lệch tín hiệu trên đường truyền. Slave hoặc master sẽ tự

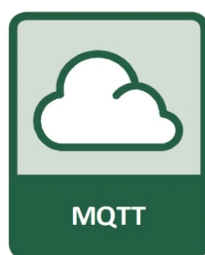
động tính toán CRC của khung tin nhận được và so sánh với giá trị CRC đi kèm để xác định tính toàn vẹn của dữ liệu.

Mặc dù đã xuất hiện nhiều giao thức truyền thông công nghiệp hiện đại hơn, Modbus RTU vẫn giữ vai trò quan trọng trong thực tế nhờ độ ổn định, tính phổ biến và khả năng tương thích rộng rãi. Việc hiểu rõ nguyên lý hoạt động và cấu trúc khung tin Modbus RTU là nền tảng quan trọng cho việc thiết kế, triển khai và vận hành các hệ thống giám sát và điều khiển công nghiệp.

3.7.3 Giao thức truyền tin MQTT/HTTP

Trong các hệ thống Internet of Things (IoT), giao thức truyền tin đóng vai trò then chốt trong việc kết nối các thiết bị phân tán với nền tảng trung tâm, đảm bảo dữ liệu được truyền tải ổn định, hiệu quả và phù hợp với đặc thù của môi trường triển khai. Trong số các giao thức phổ biến hiện nay, MQTT và HTTP là hai giao thức được sử dụng rộng rãi nhờ tính linh hoạt, khả năng tương thích cao và sự hỗ trợ mạnh mẽ từ cộng đồng.

a. Giao thức MQTT



Hình 3.6: Giao thức MQTT

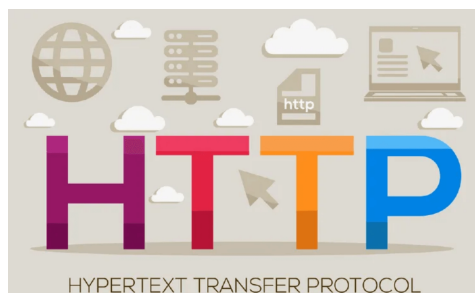
MQTT (Message Queuing Telemetry Transport) là một giao thức truyền tin nhẹ, được thiết kế theo mô hình publish-subscribe, đặc biệt phù hợp cho các hệ thống IoT có tài nguyên hạn chế và yêu cầu truyền dữ liệu với độ trễ thấp. Trong mô hình này, các thiết bị không giao tiếp trực tiếp với nhau mà thông qua một thành phần trung gian gọi là broker. Thiết bị gửi dữ liệu sẽ đóng vai trò publisher, trong khi thiết bị nhận dữ liệu là subscriber, và broker chịu trách nhiệm phân phối thông điệp dựa trên các chủ đề (topic).

Một đặc điểm nổi bật của MQTT là kích thước gói tin nhỏ, giúp giảm băng thông và tiết kiệm năng lượng, rất phù hợp với các thiết bị nhúng và mạng truyền thông không ổn định. MQTT hỗ trợ nhiều mức chất lượng dịch vụ (Quality of Service – QoS), cho phép cân bằng giữa độ tin cậy và hiệu năng truyền thông. Bên cạnh đó, giao thức còn

hỗ trợ cơ chế giữ kết nối (keep-alive), thông báo trạng thái thiết bị và khả năng mở rộng linh hoạt trong các hệ thống IoT quy mô lớn.

Nhờ các đặc tính trên, MQTT thường được sử dụng trong các ứng dụng giám sát thời gian thực, thu thập dữ liệu cảm biến, hệ thống điều khiển từ xa và các nền tảng IoT trung tâm như CoreIoT hoặc các nền tảng tương đương.

b. Giao thức HTTP



Hình 3.7: Giao thức HTTP

HTTP (HyperText Transfer Protocol) là giao thức truyền thông phổ biến nhất trên Internet, hoạt động theo mô hình client-server với cơ chế yêu cầu – phản hồi (request-response). Trong hệ thống IoT, HTTP thường được sử dụng để gửi dữ liệu từ thiết bị lên máy chủ hoặc truy vấn thông tin từ nền tảng trung tâm thông qua các API.

Ưu điểm lớn của HTTP là tính phổ biến và khả năng tương thích cao, dễ triển khai trên nhiều nền tảng và ngôn ngữ lập trình khác nhau. HTTP cho phép truyền tải dữ liệu dưới nhiều định dạng như JSON hoặc XML, thuận tiện cho việc tích hợp với các hệ thống quản lý, phân tích và hiển thị dữ liệu. Tuy nhiên, do mỗi lần truyền thông đều yêu cầu thiết lập kết nối và gói tin có kích thước tương đối lớn, HTTP không tối ưu cho các ứng dụng IoT yêu cầu truyền dữ liệu liên tục với tần suất cao hoặc trong môi trường mạng hạn chế.

Trong thực tế, HTTP thường được sử dụng cho các tác vụ cấu hình hệ thống, quản lý thiết bị, truy vấn dữ liệu hoặc tích hợp với các dịch vụ web, trong khi các tác vụ truyền dữ liệu thời gian thực ưu tiên sử dụng MQTT.

MQTT và HTTP đều có vai trò quan trọng trong hệ thống IoT, nhưng phù hợp với các mục đích sử dụng khác nhau. MQTT hướng tới truyền dữ liệu nhẹ, liên tục và thời gian thực giữa các thiết bị và nền tảng trung tâm, trong khi HTTP phù hợp cho các tác vụ quản lý, cấu hình và tích hợp hệ thống. Việc kết hợp cả hai giao thức trong cùng một hệ thống giúp tận dụng ưu điểm của từng giao thức, nâng cao tính linh hoạt và hiệu quả vận hành.

Trong khuôn khổ đề tài của nhóm, MQTT và HTTP được xem là các giao thức truyền tin chủ đạo để kết nối thiết bị nhúng với nền tảng CoreIoT, đáp ứng yêu cầu

truyền dữ liệu cảm biến, giám sát hệ thống và quản lý thiết bị một cách hiệu quả và ổn định.

3.8 Kết nối Cloud và IoT

Chương 4

Thiết kế hệ thống

Chương này tập trung vào thiết kế 3 thành phần kỹ thuật cốt lõi của nền tảng: Driver phần cứng, Bảo mật hệ thống và Công cụ hỗ trợ phát triển.

- 4.1 Kiến trúc tổng thể**
- 4.2 Thiết kế Device Driver**
- 4.3 Thiết kế Bảo mật Hệ thống**
- 4.4 Thiết kế Configuration Tool**

Chương 5

Triển khai và đánh giá

Chương này trình bày quá trình cài đặt, tích hợp và kết quả kiểm thử từng thành phần kỹ thuật của nền tảng.

5.1 Môi trường phát triển

5.2 Hiện thực Device Driver

5.3 Hiện thực Bảo mật Hệ thống

5.4 Hiện thực Configuration Tool

Chương 6

Sản phẩm demo tích hợp

Chương này là minh chứng thực nghiệm cho toàn bộ nền tảng: một hệ thống tự động hóa công nghiệp hoàn chỉnh được xây dựng trên nền tảng đã phát triển ở các chương trước.

- 6.1** Tổng quan sản phẩm demo
- 6.2** Kiến trúc hệ thống demo
- 6.3** Thiết kế và triển khai từng module
- 6.4** Tích hợp và kiểm thử sản phẩm demo

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

7.2 Hướng phát triển